

PsiloData — Dataset ds006072: adquisición y preparación BIDS

Adquisición y preparación BIDS de ds006072 (rama psilocibina)

Juan

2026-06-03

Table of contents

1	1. Dataset ds006072 (Psilocybin Precision Functional Mapping)	2
1.1	1.1 Adquisición de datos (OpenNeuro ds006072)	2
1.1.1	1.1.1 Elección del dataset	2
1.2	1.2 Descripción del estudio original	3
1.3	1.3 Parámetros de adquisición	4
1.4	1.4 Datos conductuales y fenotípicos	5
1.5	1.5 Estructura del dataset (BIDS)	6
2	2. Descarga selectiva	8
2.0.1	2.1 Instalación de AWS CLI	8
2.0.2	2.2 Script de descarga	8
3	3. Selección de datos: pre-flight	10
4	4. Heterogeneidad del dataset: part-phase, convención de runs y frames truncados	24
4.0.1	4.1 part-phase solo publicada para sub-P4 a sub-P7	24
4.0.2	4.2 Convención BIDS variable: BOLDREST{1,2} × run-{1,2} vs BOLDREST{1..6}	25
4.0.3	4.3 Runs truncados y frames de ruido	25
4.0.4	4.4 T1w multi-echo (MEMPRAGE 4-eco) sólo en sub-P7	26
5	5. Correcciones del dataset ds006072	28
5.1	5.1 Creación de participants.tsv	28
5.2	5.2 Corrección de metadatos BIDS: campo TaskName	29
5.2.1	5.2.1 Descripción del problema	29
5.2.2	5.2.2 Causa de la discrepancia	30

5.2.3	5.2.3 Justificación de la corrección	30
5.2.4	5.2.4 Corrección automática	30
5.3	5.3 Corrección de metadatos BIDS: campos <code>B0FieldIdentifier</code> / <code>B0FieldSource</code>	33
5.3.1	5.3.1 Descripción del problema	33
5.3.2	5.3.2 Estrategia de corrección	34
5.3.3	5.3.3 Corrección automática	34
5.4	5.4 Correcciones para sub-P7	38
5.4.1	5.4.1 Descartar T1w multi-echo	38
5.4.2	5.4.2 Homogeneizar resolución T1w a 0.9 mm	39
5.4.3	5.4.3 Independencia de NORDIC	44
6	6. Estrategia incremental de procesamiento: Tier 0	44
6.1	6.1 Runs excluidos permanentemente	45
6.2	6.2 Selección Tier 0 — primer pase del pipeline	45
6.3	6.3 Runs en reserva — tiers posteriores	47
6.4	6.4 Recuento global y trade-off metodológico	47
6.5	6.5 Operacionalización: dataset filtrado por tier en <code>prep/nordic/data/</code>	48

1 1. Dataset ds006072 (Psilocybin Precision Functional Mapping)

El dataset **ds006072** en OpenNeuro (Siegel et al., 2024) contiene datos **genuinamente crudos** en formato BIDS con *fieldmaps*.

1.1 1.1 Adquisición de datos (OpenNeuro ds006072)

1.1.1 Elección del dataset

Se adopta como dataset principal el **ds006072** publicado en [OpenNeuro](#), que contiene los datos del estudio “*Psilocybin desynchronizes the human brain*” de Siegel et al. (2024), publicado en *Nature* (Vol. 632, pp. 131–138). Este dataset proporciona datos **genuinamente crudos** en espacio nativo del escáner (`DatasetType`: “`raw`”), lo que permite ejecutar correctamente el pipeline completo de preprocesamiento y denoising.

La elección de ds006072 se justifica por:

1. **Datos crudos en espacio nativo:** imágenes T1w y T2w a 0.9 mm isotrópico, BOLD multi-echo sin procesar, y *fieldmaps* de spin-echo para corrección de distorsión por susceptibilidad, todo en formato BIDS v1.6.0.
2. **Diseño cruzado controlado:** estudio aleatorizado *crossover* con psilocibina (25 mg) vs. metilfenidato (40 mg, placebo activo), lo que permite la comparación intra-sujeto droga vs. control.

3. **Compatibilidad con fMRIPrep:** al disponer de datos en espacio nativo, fMRIPrep puede estimar correctamente parámetros de movimiento, confounds tisulares (aCompCor, tCompCor) y *framewise displacement*.
4. **Relevancia científica:** publicado en *Nature* en 2024, el estudio demuestra que la psilocibina produce una desincronización masiva de las redes cerebrales funcionales, con los mayores efectos en la *default-mode network* (DMN), y una reducción persistente de la conectividad entre el hipocampo anterior y la DMN durante semanas tras la administración.
5. **Licencia abierta:** CC0 / PDDL.

1.2 Descripción del estudio original

Siete voluntarios sanos (edad media 34.1 ± 9.8 años; 3 mujeres, 6 caucásicos) participaron en un estudio cruzado aleatorizado con psilocibina (25 mg oral) y metilfenidato (40 mg oral, placebo activo equiparado en efectos de arousal). Los criterios de inclusión requerían exposición previa a psicodélicos (pero no en los últimos 6 meses), edad entre 18 y 45 años, y ausencia de condiciones psiquiátricas activas. El ensayo está registrado en ClinicalTrials.gov (NCT04501653).

El diseño experimental (*precision functional mapping*) consistió en:

1. **Baseline extenso:** múltiples sesiones de imagen previas a la administración de droga (4–9 sesiones por sujeto), para establecer la variabilidad basal individual y familiarizar a los participantes con el escáner.
2. **Sesiones de droga:** imagen iniciada 60–180 minutos después de la administración de cada sustancia, durante el pico de efectos subjetivos.
3. **Seguimiento longitudinal:** imagen periódica hasta 2–3 semanas después de cada droga, para evaluar efectos persistentes.
4. **Replicación:** cuatro participantes (P1, P3, P4, P5) completaron un protocolo *open-label* de replicación con psilocibina >6 meses después.

El orden de administración fue balanceado (4 sujetos recibieron MTP primero, 3 recibieron PSIL primero), con ~1–2 semanas de separación entre drogas.

Table 1: Diseño crossover y calendario de sesiones por sujeto en el estudio original. La sesión entre paréntesis indica la de cada droga; «Sesiones totales» abarca baseline, sesiones de droga y seguimiento. sub-P2 queda excluido del análisis de este TFM (ver el callout siguiente).

Sujeto	Droga 1	Droga 2	Sesiones totales
P1	MTP (ses-6)	PSIL (ses-11)	19
P2	PSIL (ses-10)	MTP (ses-16)	21
P3	MTP (ses-8)	PSIL (ses-12)	18
P4	MTP (ses-6)	PSIL (ses-10)	15

Sujeto	Droga 1	Droga 2	Sesiones totales
P5	PSIL (ses-6)	MTP (ses-10)	14
P6	MTP (ses-5)	PSIL (ses-9)	12
P7	PSIL (ses-7)	MTP (ses-11)	14

! Exclusión de sub-P2

Siegel et al. (2024) excluyeron a P2 de los análisis de conectividad funcional: no toleró la fMRI bajo psilocibina y tuvo dificultades para mantenerse despierto en numerosas sesiones posteriores a la dosis, de modo que sólo lo conservaron para las métricas de calidad de datos. Este TFM aplica el mismo criterio y excluye a sub-P2 desde el inicio—no se preprocesa ni entra en ningún análisis—, por lo que la muestra de trabajo es $N = 6$ (sub-P1, sub-P3, sub-P4, sub-P5, sub-P6, sub-P7). En esta muestra el orden de administración se reparte en 4 sujetos con metilfenidato primero (P1, P3, P4, P6) y 2 con psilocibina primero (P5, P7).

Los hallazgos principales incluyen: (a) la psilocibina produjo cambios en conectividad funcional >3 veces mayores que el metilfenidato, (b) estos cambios fueron impulsados por la desincronización cerebral a múltiples escalas espaciales, disolviendo las distinciones entre redes, (c) los cambios individuales en FC correlacionaron fuertemente con la experiencia subjetiva mística (MEQ30, $r^2 = 0.81$), y (d) se observó una reducción persistente de la conectividad hipocampo anterior-DMN durante 3 semanas post-psilocibina, un posible correlato del efecto neuroplástico.

1.3 1.3 Parámetros de adquisición

Las adquisiciones se realizaron en un escáner **Siemens Prisma** con los siguientes parámetros:

Modalidad	Parámetro	Valor
T1w / T2w BOLD (resting state)	Resolución	0.9 mm isotrópico
	Secuencia	EPI multi-echo (5 ecos)
	Resolución	2 mm isotrópico
	Tiempos de eco (TE)	14.20, 38.93, 63.66, 88.39, 113.12 ms
	Tiempo de repetición (TR)	1761 ms
	Ángulo de flip	68°
	Aceleración in-plane	IPAT/GRAPPA = 2
	Aceleración multibanda (SMS)	6
	Cobertura	72 cortes axiales (144 mm)
BOLD (task)	Frames por run	513 nominales (~15 min); ver §1.5
	Frames por run	233 (~6 min 50 s)

Modalidad	Parámetro	Valor
Fieldmaps	Tipo	Spin-echo AP/PA

La adquisición multi-echo permite la separación de señal BOLD (dependiente de $T2^*$) de artefactos no-BOLD mediante herramientas como TEDANA, y es compatible con el pipeline de fMRIPrep.

💡 Multi-echo y denoising posterior

Los 5 tiempos de eco (14.20–113.12 ms) habilitan el uso de **TEDANA** (*TE-Dependent ANAlysis*) como estrategia de denoising posterior a fMRIPrep. TEDANA separa los componentes de señal BOLD (dependientes de $T2^*$) de los artefactos no-BOLD (movimiento, señal fisiológica) a partir de la dependencia del contraste con el tiempo de eco — un criterio físico, no estadístico. fMRIPrep preprocesa cada eco por separado con la opción `--me-output-echos`, y TEDANA se aplica como paso posterior sobre las imágenes preprocesadas. El fundamento de esta estrategia y su comparación con los métodos de denoising monoeco se desarrollan en PsiloPrep §5; en lo que toca a la elección del dataset, basta señalar que la adquisición multi-echo de ds006072 hace posible esta vía, lo que pesa de forma decisiva dado el reducido tamaño muestral ($N = 6$), donde maximizar la calidad de cada observación es crítico.

1.4 1.4 Datos conductuales y fenotípicos

El dataset incluye dos archivos de datos conductuales en `BehavioralAssessments/`:

- **PPFM_MEQ_DATA.xlsx**: puntuaciones del *Mystical Experience Questionnaire* (MEQ30) de 30 ítems, que mide la intensidad de la experiencia mística (sentimientos de conexión, trascendencia del tiempo y espacio, asombro). Puntuación máxima: 150.
- **Psilocybin-NonIdentifiableInfor_DATA_*.csv**: datos no identificables de REDCap con 165 registros y >300 variables, incluyendo:
 - Cuestionario pre-scan de neuroimagen (sueño, cafeína, medicación, ejercicio)
 - *Emotional Breakthrough Inventory* (EBI)
 - *Mini-IPIP* (personalidad Big Five, 20 ítems)
 - *Dimensional Change Card Sort* (flexibilidad cognitiva)
 - Formularios de monitorización del día de dosificación (efectos adversos, náusea, cefalea)
 - Notas de visita

Adicionalmente, `PPFM_session_notes.xlsx` documenta incidencias de cada sesión de MRI (sueño durante el escaneo, movimiento excesivo, fallos del escáner, problemas con la cinta respiratoria).

1.5 Estructura del dataset (BIDS)

El dataset sigue el estándar BIDS v1.6.0 (DatasetType: "raw", DOI: 10.18112/openneuro.ds006072.v1.0.8). Contiene 7 sujetos del protocolo principal (sub-P1 a sub-P7; la muestra de análisis de este TFM excluye a sub-P2, ver §1.2) y 4 sujetos de replicación (sub-P1R, sub-P3R, sub-P4R, sub-P5R), con un total de ~130 sesiones de MRI. La estructura varía entre sujetos (dirección de fase, número de runs, presencia de imágenes **part-phase**):

```
ds006072/
  BehavioralAssessments/      # Datos conductuales (MEQ, REDCap)
  NON_BIDS/                  # CIFTIs procesados (no se utilizan)
  dataset_description.json
  session_data.csv           # Mapeo sesión → condición (Baseline/Drug/Between/After)
  README
  sub-{P}/
    ses-{N}/
      anat/                  # Solo en sesión baseline (ses-1; ses-0 para P7)
        *_T1w.nii.gz        # T1w 0.9 mm (1-2 runs; P7: multi-echo × 6 runs)
        *_T2w.nii.gz        # T2w 0.9 mm (1-2 runs; P7: 6 runs)
      fmap/                  # Fieldmaps spin-echo AP + PA (1-2 runs)
        *_acq-SpinEchoFieldMap{AP,PA}_dir-{AP,PA}_run-{N}_epi.nii
        (+ JSON sidecars con IntendedFor y PhaseEncodingDirection)
      func/                  # BOLD multi-echo resting state
        *_task-BOLDREST{1..6}_dir-{AP|PA}_run-{1,2}_echo-{1..5}_part-
mag_bold.nii
        *_task-BOLDREST{1..6}_dir-{AP|PA}_run-{1,2}_echo-{1..5}_part-
phase_bold.nii # solo P4-P7
        (+ JSON sidecars; mag siempre + phase si HAS_PHASE; ver §4.1)
      dwi/                  # Difusión (no se utiliza ni se descarga)
```

La estructura real en S3 por sesión de droga, verificada mediante `aws s3 ls` (cada línea corresponde a ~15 min de adquisición resting-state, 513 frames nominales por run; el desglose entre frames analíticos y de ruido se detalla bajo la tabla):

Table 3: Adquisiciones resting-state por sesión de droga (verificado en S3). † = escaneo truncado o abortado (menos frames que el nominal del run; el pre-flight de §3 fija el veredicto run a run). ‡ = 516 frames (513 analíticos + 3 de ruido al final, en lugar de su posición estándar). La convención BIDS cambia entre sujetos: P1/P3/P5 usan BOLDREST1/BOLDREST2 × run-1/run-2; P4/P6/P7 usan BOLDREST1...BOLDREST6 con un único run-1. Ambas convenciones representan adquisiciones independientes equivalentes (~15 min cada una).

Sujeto	Sesión MTP	Tareas MTP	Sesión PSIL	Tareas PSIL	dir	part-phase
P1	ses-6	REST1/2 × run-1/2 (4)	ses-11	REST1/2 × run-1/2 (4)	AP	–
P2	ses-16	REST1, REST2 (2)	ses-10	REST1, REST2 (2)	AP	–
P3	ses-8	REST1/2 × run-1/2 (4)	ses-12	REST1/2 × run-1/2 (4)	AP	–
P4	ses-6	REST1, 2, 3 (3)	ses-10	REST1, REST2 (2)	PA	
P5	ses-10	REST1–4 × run-1/2 (8)	ses-6	REST1–4 × run-1/2 (8)	PA	
P6	ses-5	REST1, 2, 3†, 4 (4)	ses-9	REST1‡, 2‡, 3, 4 (4)	PA	
P7	ses-11	REST1, 2, 3 (3)	ses-7	REST1–6 (6)‡	PA	

i Conteo de frames por run

El run resting-state nominal consta de **513 frames**, pero su composición no es uniforme entre sujetos; el pre-flight (§3) la fija run a run a partir de las cabeceras NIFTI:

- En las adquisiciones *complex-valued* (sub-P4 a sub-P7) los **3 últimos son frames de ruido** para NORDIC, lo que deja **510 frames analíticos**.
- En las adquisiciones *magnitude-only* (sub-P1 a sub-P3) el pre-flight no detecta frames de ruido separables: los **513 frames son analíticos**.
- sub-P7 ses-7 es la excepción: **516 frames** adquiridos (513 analíticos + 3 de ruido), con los de ruido al final en lugar de su posición estándar.

Por eso la cifra de «510 frames + 3 de ruido» que recoge Siegel et al. (2024) describe las sesiones *complex-valued*, pero no es válida para todos los sujetos. El conteo definitivo es el de `preflight_func.csv` (§3).

2 2. Descarga selectiva

Para el pipeline de análisis de conectividad funcional (fMRIPrep → denoising → conectividad) se necesitan:

- **Anatomía (anat/)**: T1w y T2w de una sesión baseline por sujeto (para segmentación, registro y reconstrucción de superficies con FreeSurfer)
- **Fieldmaps (fmap/)**: de las sesiones de droga (para corrección de distorsión por susceptibilidad, SDC)
- **REST BOLD (func/)**: todas las tareas BOLDREST* publicadas para cada sesión de droga (de 2 a 8 adquisiciones por sesión, ver la tabla de §1.5), **part-mag** (magnitud) para todos los sujetos y **part-phase** (fase) **solo para P4–P7** (P1–P3 carecen de fase publicada en OpenNeuro). NORDIC se aplica en modo complex-valued cuando hay fase y en modo magnitude-only cuando no la hay; ambos modos están validados (Moeller et al., 2021; Vizioli et al., 2021) y son consistentes con la estrategia descrita en Siegel et al. (2024). Detalle en PsiloPrep §7.4.
- **Metadatos raíz**: `dataset_description.json`, `session_data.csv`, `README`, `BehavioralAssessments/`

No se necesitan: task BOLD (BOLDTASK), difusión (DWI), sujetos de replicación (sub-P1R, sub-P3R, sub-P4R, sub-P5R), CIFTIs procesados (NON_BIDS/), ni las decenas de sesiones baseline y follow-up.

2.0.1 2.1 Instalación de AWS CLI

```
# En PowerShell (Windows):  
winget install Amazon.AWSCLI  
# Cerrar y reabrir la terminal tras la instalación  
aws --version
```

2.0.2 2.2 Script de descarga

Los archivos funcionales en ds006072 están en formato NIfTI sin comprimir (.nii), lo que resulta en ~830 MB por eco. Para gestionar el espacio en disco, se ha desarrollado un script que descarga sesión por sesión y **comprime cada archivo a .nii.gz inmediatamente** tras la descarga (compresión *lossless* gzip, sin pérdida de información), reduciendo el tamaño ~1.3–1.6× (ratio inferior al habitual en fMRI single-echo). El script **descubre dinámicamente las tareas BOLDREST* publicadas** en cada sesión vía `aws s3 ls` (la función `list_s3_rest_tasks`), de modo que absorbe la convención BIDS variable de `task/run` entre sujetos (REST1/REST2 × run-1/run-2 en P1/P3/P5, REST1–REST6 × run-1 en P4/P6/P7; ver §4) sin codificarla a

mano. El modo bulk descarga las tareas BOLDREST1–BOLDREST4 (tope MAX_REST); BOLDREST5 y BOLDREST6, presentes sólo en sub-P7/ses-7, se recuperan a demanda con el modo single. Descargar todas las adquisiciones de resting state disponibles por sesión de droga maximiza la potencia estadística dado el reducido tamaño muestral ($N = 6$ sujetos de análisis; se descargan los 7 publicados y sub-P2 se excluye después, ver §1.2). Se descargan las imágenes de magnitud (part-mag) para todos los sujetos y, **cuando están publicadas (P4–P7)**, también las de fase (part-phase). El script controla este toggle por sujeto vía la tabla HAS_PHASE. NORDIC consume la fase para construir la representación compleja en P4–P7 (gold standard) y opera en modo magnitude-only en P1–P3 (Moeller et al., 2021; Vizioli et al., 2021).

El script se encuentra en `src/bash/download_ds006072.sh` y admite dos modos.

Modo bulk (por defecto, sin argumentos) — descarga la matriz completa $P1..P7 \times \text{DRUG1/DRUG2} \times \text{BOLDREST1..BOLDREST}\{\text{MAX_REST}\}$ con `MAX_REST = 4`:

```
bash D:/ProfessionalProyectos/PsyConn/src/bash/download_ds006072.sh
```

Modo single (`--sub`, `--ses`, `--task`, opcional `--run`) — descarga **un único BOLD específico** (part-mag + part-phase cuando aplica) junto con el `anat` de referencia del sujeto y los `fmap` de la sesión target. Está pensado para recuperar adquisiciones que quedan fuera del tope MAX_REST sin reabrir el scope global del bulk: el caso canónico es BOLDREST5/BOLDREST6 de sub-P7/ses-7, excluidos por viabilidad de NORDIC + fMRIPrep + TEDANA en el diseño del bulk pero recuperables a demanda para experimentos puntuales (ver §4). El modo single comparte con el bulk las mismas funciones de compresión (`compress_nii`), verificación de completitud (`is_complete`) y la tabla HAS_PHASE que decide complex- vs magnitude-only NORDIC. Es idempotente: re-ejecutar la misma invocación no re-descarga lo que ya está en disco y comprimido.

```
# Caso de uso: descargar sub-P7 ses-7 BOLDREST6 run-1 (fuera del tope bulk)
bash D:/ProfessionalProyectos/PsyConn/src/bash/download_ds006072.sh \
  --sub P4 --ses 10 --task BOLDREST3 --run 1
```

Si se omite `--run`, el patrón no restringe el número de run y se bajan todos los runs publicados de esa task para la sesión indicada.

En **modo bulk** el script realiza las siguientes operaciones:

0. Descarga metadatos raíz (`dataset_description.json`, `README`, `session_data.csv`, `BehavioralAssessments/`, etc.).
1. Para cada sujeto (P1–P7), descarga la sesión de anatomía (T1w + T2w a 0.9 mm) y la comprime.

2. Para cada sesión de droga (PSIL y MTP): descarga fieldmaps (spin-echo AP/PA) y comprime; luego **descubre dinámicamente las tareas BOLDREST*** publicadas en S3 (`list_s3_rest_tasks`) y, para cada una, descarga y comprime `part-mag` (5 ecos) y, si `HAS_PHASE[$SUB]=1`, también `part-phase` (5 ecos) — liberando espacio antes de cada paso.
3. Salta automáticamente componentes ya descargados y comprimidos (idempotencia por componente: cada `part-mag/part-phase` de cada BOLDREST* se verifica de forma independiente). Para P1–P3 imprime `[SKIP] ses-N BOLDRESTk part-phase (sub-Pn sin fase publicada -> NORDIC magnitude-only)`.
4. Es reanudable: si se interrumpe, re-ejecutar el script continúa donde se quedó.
5. Al finalizar, muestra un resumen de espacio por sujeto, etiqueta el modo NORDIC esperado (`complex-NORDIC` vs `mag-only-NORDIC`) y verifica que el número total de adquisiciones `part-mag` y `part-phase` coincidan **solo para los sujetos con fase** (P4–P7), y que no queden `.nii` sin comprimir.

En **modo single** el script reutiliza el paso 0 (metadatos) y a continuación, en vez de iterar P1..P7, ejecuta un bloque dirigido sobre los argumentos `--sub/--ses/--task/--run`: (i) descarga y comprime el `anat` de referencia del sujeto si falta, (ii) descarga y comprime los `fmap` de la sesión target si faltan, (iii) descarga y comprime `part-mag` para el BOLDREST* solicitado (filtrado por `_run-N_` cuando se da `--run`), (iv) descarga y comprime `part-phase` solo si `HAS_PHASE[$SUB]=1`, y (v) imprime un resumen específico (`anat`, `fmap`, `mag`, `phase`, tamaño de la sesión y etiqueta de modo NORDIC) y sale con código 1 si no se encontró ningún `part-mag` (típicamente porque la combinación pedida no existe en S3).

3 3. Selección de datos: pre-flight

Tras la descarga (§2) y antes de aplicar cualquier corrección, copia incremental o denoising, ejecutamos un pre-flight que recorre el dataset entero. Los runs marcados como `EXCLUDED` se derivan de dos criterios: el paramétrico `Q_USABLE_MIN` —runs demasiado cortos para aportar a la conectividad funcional tras el censoring por movimiento (ver el check 5)— y la exclusión a nivel de sujeto de sub-P2, fuera del análisis desde el inicio (§1.2). Los sujetos `magnitude-only` se derivan de la presencia o ausencia de `_part-phase_*.nii.gz` en disco. Los dos CSV resultantes son el registro de validación del dataset: las decisiones de §5 (correcciones BIDS), §6 (selección de Tier 0) y §7 (NORDIC) deben mantenerse coherentes con ellos.

El pre-flight produce dos CSVs (`preflight_func.csv` para los runs BOLD, `preflight_anat.csv` para las sesiones anatómicas) con un veredicto `PASS / WARN / FAIL / EXCLUDED` por entrada y un *reason* legible. Cada veredicto consolida una batería de checks atómicos descritos abajo.

Supuestos evaluados por run BOLD (12 checks):

1. Los 5 ecos del protocolo están presentes (`echo-1 ... echo-5`).

2. **Integridad del gzip + header NIfTI + array 4D** para los ficheros `_part-mag_bold.nii.gz` (y `_part-phase_bold.nii.gz` si existe).
3. **Consistencia espacio-temporal entre los 5 ecos**: misma shape 4D (X, Y, Z, T).
4. **Número de frames $Q \geq Q_MIN$** — mínimo técnico (50) para PCA con el parche 11^3 de NIFTI_NORDIC.m. Q por debajo $\rightarrow$ FAIL (NORDIC no puede ejecutarse).
5. **Número de frames $Q \geq Q_USABLE_MIN$** — mínimo de frames (400) para que un run aporte a la conectividad funcional tras el censoring por movimiento. La fiabilidad de la FC individual crece con la cantidad total de datos (Laumann 2015); Q_USABLE_MIN descarta los runs demasiado cortos para contribuir, mientras que la estabilidad por sujeto se alcanza concatenando runs. Q por debajo de este umbral analítico pero por encima del técnico $\rightarrow$ EXCLUDED con `reason` que cita el criterio.
6. **Dtype y escalado BIDS válidos** — `scl_slope/scl_inter` sin valores Inf; NaN es válido (en NIFTI-1 equivale a «sin escalado aplicado»).
7. **Sin NaN/Inf** en los frames muestreados (primero, medio, último).
8. **Detección empírica de bloque de ruido terminal**: `ratio < 0.20` $\rightarrow$ noise presente; `> 0.50` $\rightarrow$ ausente; entre ambos $\rightarrow$ FAIL por ambigüedad.
9. **Sidecar JSON presente y coherente**: `RepetitionTime` y `TaskName` consistentes entre ecos; `B0FieldSource` presente (WARN si falta — alerta para §5.3, no bloquea NORDIC); `TaskName` igual a la entidad `task-<label>` del filename (WARN si el valor es el DICOM-derived sucio — alerta para §5.2).
10. **Idempotencia**: `DenoisingMethod != "NORDIC"` en el sidecar (el run no ha sido procesado previamente).
11. **Clase NORDIC descubierta**: el modo (`complex` vs `magnitude-only`) se asigna por sujeto según si existe al menos un `_part-phase_*.nii.gz` en su `func/`. En modo `complex`, comprueba además que cada eco tiene fase emparejada con la misma shape que la magnitud; en modo `magnitude-only`, WARN si aparece fase inesperada.
12. **Dir AP/PA registrado** por run (sólo informativo en el CSV; no bloquea ningún veredicto).

Supuestos evaluados por sesión anatómica (3 checks):

1. **Presencia de al menos un `*_T1w.nii.gz`** en `sub-PX/ses-*/anat/` (FAIL si ausente; fMRIPrep no puede construir el template anatómico).
2. **Ausencia de ecos T1w individuales** (MEMPRAGE multi-echo). El pre-flight calcula sobre el dataset si el patrón modal tiene o no ecos individuales y marca WARN cualquier sesión que se desvíe — alerta para el pre-flight de §5.4.1. El número de `*_T1w.nii.gz` simples (1 vs 2) es variación BIDS normal y no se marca.
3. **Resolución T1w homogénea** (zooms del header). Cualquier sesión cuyos zooms difieran del modal del dataset se reporta WARN — alerta para §5.4.2.

Además, el chunk resuelve y registra el **plan NORDIC por run** (`noise_volume_last, n_trimmed, mode`) y produce los CSVs `preflight_func.csv` + `preflight_anat.csv`. Esos CSV son el registro de validación del dataset: las decisiones de §5, §6 y §7 deben verificarse frente a ellos.

Sujeto	Sesión	Runs	Frames	Interpretación
P6	ses-5	BOLDREST3	245	Scan terminado prematuramente; aunque el README menciona volúmenes de ruido copiados de un scan vecino, el pre-flight no detecta un bloque de ruido terminal (<code>has_noise=False</code>) y los 245 frames se cuentan como analíticos. Excluido por $Q < 400$
P6	ses-9	BOLDREST1, BOLDREST2	208, 135	Scans abortados, sin noise volumes al final
P7	ses-7	BOLDREST1...6 (6 runs)	516 ($\times 6$)	3 frames analíticos extra ($513 + 3$ noise al final)
P7	ses-0	(anat)	—	24 echos T1w individuales + 1 RMS combine vs 1-2 T1w simples en P1-P6 ses-1

```
# =====
# Pre-flight ds006072: descubrimiento + validación.
#
# Enumera runs BOLDREST*, sesiones anatómicas y disponibilidad de
# fase por sujeto; aplica 12 checks por run BOLD y 3 checks por sesión anat;
# y produce dos CSVs (preflight_func.csv, preflight_anat.csv) que registran
# la validación del dataset; §5 (correcciones BIDS), §6 (selección de tier)
# y §7 (NORDIC) deben mantenerse coherentes con ellos.
#
# Las decisiones que toma este pre-flight son TODAS empíricas; los parámetros
# del método (umbrales Q_MIN, Q_USABLE_MIN, NOISE_RATIO_*) son del método,
# no resultados del análisis. Reutiliza la lógica de:
#   - inspect_nordic_anomalies.py (integridad de fichero)
#   - detect_noise_volumes.py      (ratio terminal/central)
#
# Dependencias: nibabel, numpy, pandas (no oct2py; no modifica nada).
# =====
import os, re, json, gzip, functools
from collections import Counter, defaultdict
from pathlib import Path

import numpy as np
import pandas as pd
import nibabel as nib
```

```

print = functools.partial(print, flush=True)

# -----
# 0. Parámetros del método (NO supuestos sobre el dataset)
# -----
BIDS_DIR      = "F:/JIVillL/openneuro/ds006072"
SUBJECTS      = ["sub-P1", "sub-P2", "sub-P3", "sub-P4",
                  "sub-P5", "sub-P6", "sub-P7"]
# sub-P2 se recorre en el inventario pero queda EXCLUDED por construcción:
# excluido del análisis desde el inicio (§1.2; no toleró la fMRI bajo
# psilocibina - Siegel et al., 2024).
EXCLUDED_SUBJECTS = {"sub-P2"}
N_ECHOES      = 5
N_NOISE_VOL    = 3
Q_MIN         = 50      # mínimo técnico para PCA con parche 113 (NIFTI_NORDIC)
Q_USABLE_MIN  = 400     # mínimo de frames por run para aportar a la FC tras censoring (Laumann)
NOISE_RATIO_PRESENT = 0.20
NOISE_RATIO_ABSENT  = 0.50

DERIV_DIR     = "D:/ProfessionalProyectos/PsyConn/derivatives/ds006072/prep/pre_flight"
CSV_FUNC_OUT  = DERIV_DIR + "/preflight_func.csv"
CSV_ANAT_OUT  = DERIV_DIR + "/preflight_anat.csv"

# -----
# 1. Descubrimiento global del dataset (runs BOLD + fase + anat)
# -----
def discover_runs(bids_dir, subjects):
    """Enumera todos los runs BOLDREST* agrupados por (subj, ses, task, run)."""
    runs = defaultdict(lambda: {"echoes": {}, "dirs": set()})
    for subj in subjects:
        for p in sorted(Path(bids_dir, subj).glob(
            "ses-*/func/*_part-mag_bold.nii.gz")):
            m = re.search(
                r"(sub-P\d+)_ (ses-[\^_]+)_task-([\^_]+)_dir-([\^_]+)_ "
                r"(run-\d+)_echo-(\d+)_part-mag_bold\.nii\.gz",
                p.name)
            if not m:
                continue
            s, se, task, d, run, echo = m.groups()
            if not task.startswith("BOLDREST"):
                continue
            key = (s, se, task, run)

```

```

        runs[key]["dirs"].add(d)
        runs[key]["echoes"][int(echo)] = p
    return runs

def discover_phase_subjects(bids_dir, subjects):
    """Deriva qué sujetos tienen al menos un _part-phase_*.nii.gz en su func/.
    El modo NORDIC (complex vs magnitude-only) sale de aquí, no de una
    constante; cualquier cambio futuro del protocolo del escáner queda
    reflejado automáticamente."""
    phase = set()
    for subj in subjects:
        hits = list(Path(bids_dir, subj).glob(
            "ses-*/func/*_part-phase_bold.nii.gz"))
        if hits:
            phase.add(subj)
    return phase

def discover_anat(bids_dir, subjects):
    """Inventario por sesión anatómica: número de T1w totales, echos
    individuales y RMS combine; zooms del primer T1w legible. No asume
    estructura concreta; reporta lo que hay en disco."""
    inv = {}
    for subj in subjects:
        for anat_dir in sorted(Path(bids_dir, subj).glob("ses-*/anat")):
            ses = anat_dir.parent.name
            t1ws = sorted(anat_dir.glob(f"{subj}_{ses}*T1w.nii.gz"))
            t1w_echos = [p for p in t1ws
                          if re.search(r"_echo-\d+_T1w\.nii\.gz$", p.name)]
            t1w_combine = [p for p in t1ws if p not in t1w_echos]
            zooms = None
            for p in (t1w_combine or t1ws):
                try:
                    zooms = tuple(round(float(z), 3)
                                  for z in nib.load(str(p)).header.get_zooms()[1:3])
                    break
                except Exception:
                    continue
            inv[(subj, ses)] = {
                "n_t1w_total": len(t1ws),
                "n_t1w_echos": len(t1w_echos),
                "n_t1w_combine": len(t1w_combine),
                "zooms": zooms,
            }

```

```

    }
    return inv

# -----
# 2. Checks atómicos
# -----

def check_gzip_and_load(path: Path):
    try:
        with open(path, "rb") as f:
            gzip.decompress(f.read())
    except Exception as e:
        return False, f"gzip: {type(e).__name__}"
    try:
        img = nib.load(str(path))
        if len(img.shape) != 4:
            return False, f"not 4D ({img.shape})"
        np.asarray(img.dataobj[..., 0])
    except Exception as e:
        return False, f"nib.load: {type(e).__name__}"
    return True, "ok"

def check_dtype_scl(path: Path):
    """En NIfTI-1, scl_slope NaN == scl_slope 0 == "sin escalado aplicado"
    (spec $scl_slope). dcm2niix emite NaN cuando el DICOM no trae rescale,
    que es el caso por defecto para EPI. Sólo bloqueamos por valores Inf
    (corrupción real) o dtype totalmente inesperado."""
    img = nib.load(str(path))
    slope = float(img.header["scl_slope"])
    inter = float(img.header["scl_inter"])
    if np.isinf(slope) or np.isinf(inter):
        return False, f"scl Inf (slope={slope}, inter={inter})"
    return True, (f"dtype={img.get_data_dtype()} "
                  f"slope={slope} inter={inter}")

def check_finite_sample(path: Path):
    img = nib.load(str(path))
    nt = img.shape[3]
    for idx in (0, nt // 2, nt - 1):
        arr = np.asarray(img.dataobj[..., idx], dtype=np.float32)
        if not np.isfinite(arr).all():
            return False, f"non-finite en frame {idx}"
    return True, "ok"

```

```

def detect_noise_volumes(path: Path, n_noise=N_NOISE_VOL,
                        thr_present=NOISE_RATIO_PRESENT,
                        thr_absent=NOISE_RATIO_ABSENT):
    img = nib.load(str(path))
    nt = img.shape[3]
    if nt <= n_noise:
        raise ValueError(f"Q={nt} <= n_noise={n_noise}")
    dataobj = img.dataobj
    mid = np.asarray(dataobj[..., nt // 2], dtype=np.float32)
    mid_max = float(mid.max())
    if mid_max <= 0:
        raise ValueError("frame central vacío")
    brain = mid > 0.20 * mid_max
    if int(brain.sum()) < 1000:
        raise ValueError(f"máscara cerebral pequeña ({int(brain.sum())})")
    analytic = float(mid[brain].mean())
    last_means = [
        float(np.asarray(dataobj[..., nt - k], dtype=np.float32)[brain].mean())
        for k in range(n_noise, 0, -1)
    ]
    ratio = float(np.mean(last_means) / analytic)
    if ratio < thr_present:
        return True, ratio
    if ratio > thr_absent:
        return False, ratio
    raise ValueError(f"ratio ambiguo {ratio:.4f} "
                    f"(ni <{thr_present} ni >{thr_absent})")

# -----
# 3. Pipeline por run BOLD
# -----

def preflight_run(key, info, phase_subjects):
    subj, ses, task, run = key
    rec = {"subj": subj, "ses": ses, "task": task, "run": run,
          "dir": "|".join(sorted(info["dirs"])) or "?",
          "n_echoes": len(info["echoes"])}

    # Exclusión a nivel de sujeto: sub-P2 queda fuera del análisis (§1.2).
    # Se recorre para no perder la traza en el inventario, pero su veredicto
    # es EXCLUDED por construcción.
    if subj in EXCLUDED_SUBJECTS:
        rec["verdict"] = " EXCLUDED"

```

```

    rec["reason"] = f"{subj} excluido del análisis (§1.2)"
    return rec

errors, warns = [], []

# Check 1 - ecos completos
expected = set(range(1, N_ECHOES + 1))
missing = expected - set(info["echoes"].keys())
if missing:
    errors.append(f"ecos faltantes: {sorted(missing)}")
    rec["verdict"] = " FAIL"
    rec["reason"] = "; ".join(errors)
    return rec
mag_paths = [info["echoes"][e] for e in range(1, N_ECHOES + 1)]

# Check 2 - integridad de magnitud
for p in mag_paths:
    ok, msg = check_gzip_and_load(p)
    if not ok:
        errors.append(f"mag {p.name}: {msg}")

# Check 3/4 - shape consistencia entre ecos + Q >= Q_MIN técnico
shapes = []
for p in mag_paths:
    try:
        shapes.append(nib.load(str(p)).shape)
    except Exception:
        shapes.append(None)
valid = [s for s in shapes if s is not None]
if valid:
    if not all(s == valid[0] for s in valid):
        errors.append(f"shape inconsistente entre ecos: {valid}")
    rec["shape"] = tuple(valid[0])
    rec["n_frames"] = int(valid[0][3])
    if valid[0][3] < Q_MIN:
        errors.append(f"Q={valid[0][3]} < Q_MIN={Q_MIN} (NORDIC PCA infactible)")

# Check 5 - Q >= Q_USABLE_MIN (criterio analítico, no técnico):
#     si Q por debajo del umbral analítico pero por encima del técnico,
#     el veredicto será EXCLUDED (no FAIL) tras pasar los demás checks.
q_short = (rec.get("n_frames") is not None
            and rec["n_frames"] < Q_USABLE_MIN)

```

```

        and rec["n_frames"] >= Q_MIN)

# Check 6/7 - dtype/scl y finite
for p in mag_paths:
    ok, msg = check_dtype_scl(p)
    if not ok: errors.append(f"{p.name} dtype: {msg}")
if not errors:
    for p in mag_paths:
        ok, msg = check_finite_sample(p)
        if not ok: errors.append(f"{p.name} finite: {msg}")

# Check 8 - detección de noise volumes (sobre echo-1)
has_noise, noise_ratio = None, None
if not errors:
    try:
        has_noise, noise_ratio = detect_noise_volumes(mag_paths[0])
        rec["has_noise"] = has_noise
        rec["noise_ratio"] = round(noise_ratio, 4)
    except Exception as e:
        errors.append(f"detect_noise_volumes: {e}")

# Check 9 - sidecar JSON: coherencia, idempotencia y TaskName limpio
trs, tasknames, b0srcs = [], [], []
for p in mag_paths:
    js = Path(str(p).replace(".nii.gz", ".json"))
    if not js.exists():
        errors.append(f"{js.name}: sidecar ausente")
        continue
    try:
        meta = json.loads(js.read_text(encoding="utf-8"))
    except Exception as e:
        errors.append(f"{js.name}: JSON ilegible ({type(e).__name__})")
        continue
    trs.append(meta.get("RepetitionTime"))
    tasknames.append(meta.get("TaskName"))
    b0srcs.append(meta.get("B0FieldSource"))
    if meta.get("DenoisingMethod") == "NORDIC":
        errors.append(f"{js.name}: ya procesado (DenoisingMethod=NORDIC)")
trs_nonnull = {t for t in trs if t is not None}
if len(trs_nonnull) > 1:
    errors.append(f"TR inconsistente entre ecos: {trs}")
elif trs_nonnull:

```

```

    rec["tr"] = float(next(iter(trs_nonnull)))
if tasknames and len({t for t in tasknames if t}) > 1:
    errors.append(f"TaskName inconsistente entre ecos: {tasknames}")
if any(b is None for b in b0srcs):
    warns.append("B0FieldSource ausente (alerta para §5.3)")
if tasknames and tasknames[0] and tasknames[0] != task:
    warns.append(f"TaskName='{tasknames[0]}' != task-{task} "
                "(alerta para §5.2)")

# Check 10/11 - modo NORDIC descubierto + fase emparejada
is_mag_only = subj not in phase_subjects
mode = "magnitude-only" if is_mag_only else "complex"
rec["mode"] = mode
if not is_mag_only:
    for p in mag_paths:
        phase = Path(str(p).replace("_part-mag_", "_part-phase_"))
        if not phase.exists():
            errors.append(f"fase ausente: {phase.name}")
            continue
        ok, msg = check_gzip_and_load(phase)
        if not ok:
            errors.append(f"{phase.name}: {msg}")
            continue
        try:
            if nib.load(str(phase)).shape != nib.load(str(p)).shape:
                errors.append(f"fase shape != mag: {phase.name}")
        except Exception:
            errors.append(f"{phase.name}: header ilegible")
    else:
        for p in mag_paths:
            phase = Path(str(p).replace("_part-mag_", "_part-phase_"))
            if phase.exists():
                warns.append(f"fase inesperada en sujeto mag-only: {phase.name}")

# Resolución del plan NORDIC
if not errors and has_noise is not None:
    rec["noise_volume_last"] = N_NOISE_VOL if has_noise else 0
    rec["n_trimmed"] = N_NOISE_VOL if has_noise else 0
    rec["n_analytic"] = rec["n_frames"] - rec["n_trimmed"]

# Veredicto
if errors:

```

```

    rec["verdict"] = " FAIL"
    short = "; ".join(errors[:3])
    if len(errors) > 3:
        short += f" ... [{len(errors)-3} más]"
    rec["reason"] = short
elif q_short:
    rec["verdict"] = " EXCLUDED"
    rec["reason"] = (f"Q={rec['n_frames']} < Q_USABLE_MIN={Q_USABLE_MIN}"
                    "(insuficiente para FC estable tras censoring)")
elif warns:
    rec["verdict"] = " WARN"
    rec["reason"] = "; ".join(warns)
else:
    rec["verdict"] = " PASS"
    rec["reason"] = ""
return rec

# -----
# 4. Pipeline por sesión anatómica
# -----

def preflight_anat(key, inv_entry, modal):
    subj, ses = key
    rec = {"subj": subj, "ses": ses,
          "n_t1w_total": inv_entry["n_t1w_total"],
          "n_t1w_echos": inv_entry["n_t1w_echos"],
          "n_t1w_combine": inv_entry["n_t1w_combine"],
          "zooms": str(inv_entry["zooms"]) if inv_entry["zooms"] else None}
    errors, warns = [], []

    # Check anat-1 - al menos un T1w
    if inv_entry["n_t1w_total"] == 0:
        errors.append("sin T1w en anat/ (fMRIPrep no puede construir template)")

    # Check anat-2 - presencia de ecos T1w individuales (MEMPRAGE multi-echo).
    # El número de T1w simples (1 vs 2) es variación BIDS normal y no se marca;
    # lo que rompe el anat_template_wf de fMRIPrep es la presencia de ecos
    # individuales sin combinar, que FreeSurfer intenta registrar entre sí.
    has_echos_modal = modal["n_t1w_echos"] > 0
    has_echos_entry = inv_entry["n_t1w_echos"] > 0
    if has_echos_entry != has_echos_modal:
        warns.append(
            f"T1w con {inv_entry['n_t1w_echos']} ecos individuales "

```

```

        f"(modal del dataset: {modal['n_t1w_echos']}); MEMPRAGE "
        "multi-echo, alerta para §5.4.1")

# Check anat-3 - resolución homogénea vs modal
if (inv_entry["zooms"] and modal["zooms"]
    and inv_entry["zooms"] != modal["zooms"]):
    warns.append(
        f"zooms T1w={inv_entry['zooms']} != modal={modal['zooms']} "
        "(alerta para §5.4.2)")

if errors:
    rec["verdict"] = " FAIL"
    rec["reason"] = "; ".join(errors)
elif warns:
    rec["verdict"] = " WARN"
    rec["reason"] = "; ".join(warns)
else:
    rec["verdict"] = " PASS"
    rec["reason"] = ""
return rec

# -----
# 5. Ejecutar el pre-flight
# -----

runs = discover_runs(BIDS_DIR, SUBJECTS)
phase_subjects = discover_phase_subjects(BIDS_DIR, SUBJECTS)
anat_inv = discover_anat(BIDS_DIR, SUBJECTS)

print(f"Runs BOLD descubiertos      : {len(runs)}")
print(f"Sesiones anat descubiertas : {len(anat_inv)}")
print(f"Sujetos con fase publicada  : {sorted(phase_subjects)}")
print(f"Sujetos magnitude-only      : {sorted(set(SUBJECTS) - phase_subjects)}")
print("-" * 90)

# --- BOLD ---
records_func = []
for i, (key, info) in enumerate(sorted(runs.items()), 1):
    rec = preflight_run(key, info, phase_subjects)
    records_func.append(rec)
    extra = ""
    if "has_noise" in rec:
        extra = (f" noise={'Y' if rec['has_noise'] else 'N'} ")

```

```

        f"(r={rec['noise_ratio']:.3f}) Q={rec.get('n_frames','?')}")
    print(f"[{i:3d}/{len(runs)}] {rec['verdict']:12} "
          f"{rec['subj']:7} {rec['ses']:9} {rec['task']:10} {rec['run']:6}"
          f"{extra} {rec.get('reason','')}[:60]}")

df_func = pd.DataFrame(records_func)
os.makedirs(os.path.dirname(CSV_FUNC_OUT), exist_ok=True)
df_func.to_csv(CSV_FUNC_OUT, index=False)
print(f"\nCSV BOLD guardado: {CSV_FUNC_OUT}")

# --- ANAT (modal global → veredicto por sesión) ---
def _modal(values):
    counted = Counter(v for v in values if v is not None)
    return counted.most_common(1)[0][0] if counted else None

modal = {
    "n_t1w_total": _modal(v["n_t1w_total"] for v in anat_inv.values()),
    "n_t1w_echos": _modal(v["n_t1w_echos"] for v in anat_inv.values()),
    "n_t1w_combine": _modal(v["n_t1w_combine"] for v in anat_inv.values()),
    "zooms": _modal(v["zooms"] for v in anat_inv.values()),
}
print(f"\nModal anat detectado: {modal}")

records_anat = []
for key, inv_entry in sorted(anat_inv.items()):
    rec = preflight_anat(key, inv_entry, modal)
    records_anat.append(rec)
    print(f" {rec['verdict']:12} {rec['subj']:7} {rec['ses']:9} "
          f"T1w={inv_entry['n_t1w_total']:2d} (echos={inv_entry['n_t1w_echos']:2d}"
          f" + combine={inv_entry['n_t1w_combine']:1d}) "
          f"zooms={inv_entry['zooms']} {rec.get('reason','')}[:60]}")

df_anat = pd.DataFrame(records_anat)
df_anat.to_csv(CSV_ANAT_OUT, index=False)
print(f"\nCSV ANAT guardado: {CSV_ANAT_OUT}")

# -----
# 6. Resumen agregado
# -----

print("\n" + "=" * 70)
print("Resumen pre-flight ds006072")
print("=" * 70)

```

```

print("\nBOLD:")
by_v_func = df_func["verdict"].value_counts().to_dict()
for v in [" PASS", " WARN", " FAIL", " EXCLUDED"]:
    print(f" {v:12}: {by_v_func.get(v, 0):3d} runs")

processable = df_func[df_func["verdict"].isin([" PASS", " WARN"])]
if not processable.empty and "mode" in processable.columns:
    print("\nPlan NORDIC (runs procesables):")
    print(f" complex : {(processable['mode']=='complex').sum():3d}")
    print(f" magnitude-only: {(processable['mode']=='magnitude-only').sum():3d}")
    if "has_noise" in processable.columns:
        with_n = int(processable["has_noise"].fillna(False).sum())
        without_n = int((processable["has_noise"] == False).sum())
        print(f" con noise block (nvl=3): {with_n:3d}")
        print(f" sin noise block (nvl=0): {without_n:3d}")

flagged_func = df_func[df_func["verdict"].isin([" WARN", " FAIL", " EXCLUDED"])]
if not flagged_func.empty:
    print("\nRuns BOLD marcados (WARN/FAIL/EXCLUDED):")
    cols = ["subj", "ses", "task", "run", "verdict", "reason"]
    with pd.option_context("display.max_colwidth", 120, "display.width", 200):
        print(flagged_func[cols].to_string(index=False))

print("\nANAT:")
by_v_anat = df_anat["verdict"].value_counts().to_dict()
for v in [" PASS", " WARN", " FAIL"]:
    print(f" {v:12}: {by_v_anat.get(v, 0):3d} sesiones")

flagged_anat = df_anat[df_anat["verdict"].isin([" WARN", " FAIL"])]
if not flagged_anat.empty:
    print("\nSesiones anat marcadas:")
    cols = ["subj", "ses", "verdict", "reason"]
    with pd.option_context("display.max_colwidth", 120, "display.width", 200):
        print(flagged_anat[cols].to_string(index=False))

```

La regla de decisión es: **proceder con §5 → §6 → §7 sólo si no hay FAIL en ninguno de los dos CSVs**. Los veredictos `WARN` son admisibles y se resuelven en los chunks downstream (TaskName en §5.2, B0FieldSource en §5.3, anat heterogéneo en §5.4.1 y §5.4.2). Los veredictos `EXCLUDED` acotan el conjunto de runs procesables (`PASS` `WARN`): las listas de runs de §6 (tiers) y §7 (NORDIC) deben coincidir con ese conjunto, y `preflight_func.csv` es la referencia frente a la que se verifica esa coincidencia. Un `FAIL` inesperado indicaría corrupción

de descarga, truncamiento silencioso, reproceso accidental o una heterogeneidad de fase no prevista.

4 4. Heterogeneidad del dataset: part-phase, convención de runs y frames truncados

La estructura descargada no es uniforme entre sujetos (tablas anteriores). Cuatro peculiaridades sistemáticas merecen documentación porque afectan directamente al pipeline (selección del modo NORDIC, número de observaciones por sesión, criterios de exclusión, construcción del template anatómico) y no están explicadas ni en el README ni en los *Methods* de Siegel et al. (2024).

4.0.1 4.1 part-phase solo publicada para sub-P4 a sub-P7

Los sujetos P1–P3 se distribuyen en OpenNeuro únicamente como magnitud (**part-mag**), mientras que P4–P7 incluyen tanto magnitud como fase (**part-phase**). La causa es deducible a partir de los JSON sidecars de la propia adquisición:

Sujeto	Fecha (inferida del SeriesInstanceUID)	ProtocolName	ImageType
P1	2021-07-11	BOLD_REST1_2mm_mb6_mEPI	mag" (part-mag)
P4	2022-02-20	BOLD_REST1_NORDIC_ME5P2mm	mag, phase" (part-phase)

Entre P3 y P4 (~7 meses) la secuencia del Prisma se reconfiguró a una variante etiquetada explícitamente **NORDIC**, que activa la reconstrucción de fase en el reconstructor del escáner (el entorno **NORDIC_Raw** del CMRR requiere DICOMs tipo **M + P** emparejados). Antes de ese cambio, el escáner sólo guardaba magnitud, por lo que **la fase de P1–P3 no está sin publicar: físicamente nunca se reconstruyó**. Moeller et al. (2021) y Vizioli et al. (2021) validan NORDIC en modo magnitude-only para este caso; el pipeline asigna automáticamente el modo por sujeto (PsiloPrep §7.4).

El README afirma literalmente “*Nordic Denoising compatible: fMRI scans included 3 noise scans at the end of each run, and mag and phase images are included*”, pero la frase no refleja la diferencia de protocolo entre P1–P3 y P4–P7.

4.0.2 4.2 Convención BIDS variable: BOLDREST{1,2} × run-{1,2} vs BOLDREST{1..6}

Los *Methods* indican “*Every session included two 15-min resting-state fMRI (rs-fMRI) scans*”, pero la verificación de la estructura real en S3 muestra que el número de adquisiciones por sesión de droga oscila entre 2 (P2) y 8 (P5). Más relevante aún, la **convención BIDS empleada cambia entre sujetos tempranos y tardíos**:

- **P1, P3, P5** (y parcialmente P2): dos tareas BOLDREST1 + BOLDREST2, cada una con 1–2 entidades run- (4 u 8 adquisiciones por sesión de droga).
- **P4, P6, P7**: varias tareas BOLDREST1, BOLDREST2, ..., hasta BOLDREST6 en P7/ses-7, todas con un único run-1 (3 a 6 adquisiciones por sesión).

Ambas convenciones son válidas en BIDS y representan adquisiciones independientes equivalentes de ~15 min (513 frames por run; el desglose entre frames analíticos y de ruido depende del modo de adquisición — ver §1.5). El paso del esquema *task × run* al esquema *task-enumerada* es coherente con el cambio de protocolo del Prisma descrito arriba (P4 marca la transición a la secuencia NORDIC-enabled) y con pistas indirectas en la documentación:

- README: “*The number of non-drug MRI sessions was dependent on availability of the participant, scanner and scanner support staff.*”
- README: “*Any rest scans that were ended early for any reason (e.g. P6 ses-5 REST3, P4R ses-2 REST3) will not have the 3 noise images at the end.*”
- Paper: “*One participant (P2) was not able tolerate fMRI while on psilocybin, and had trouble staying awake on numerous fMRI visits after psilocybin and was thus excluded from analysis.*”
- CHANGES v1.0.7–v1.0.8 (feb-2026): “*Added missing P1 ses1 echo 1 timeseries; added missing P1 ses19 echo 2 timeseries; added missing files.*”

El pipeline trata cada adquisición (combinación **task × run**) como observación independiente, de modo que la convención BIDS concreta no afecta al análisis: los modelos ponderan por sujeto/sesión (PsyConn §6) y los contrastes PSIL vs MTP son intra-sujeto.

4.0.3 4.3 Runs truncados y frames de ruido

Cuatro situaciones se desvían de la nominal de 513 frames por run (verificado con `nibabel`):

Archivo	Frames	Interpretación
sub-P6/ses-5/.../task-BOLDREST3_run-1	245	Truncado (mencionado en README)
sub-P6/ses-9/.../task-BOLDREST1_run-1	208	Truncado (no documentado explícitamente)

Archivo	Frames	Interpretación
sub-P6/ses-9/.../task-BOLDREST2_run-1	135	Truncado (no documentado explícitamente)
sub-P7/ses-7/.../task-BOLDREST{1..6}_run-1	516	Excepción: 3 frames más que la nominal — 513 analíticos + 3 de ruido al final (ver §1.5)

Los tamaños de los archivos `.nii` truncados en S3 coinciden con los locales (~362 MB y ~235 MB para los dos runs de P6/ses-9 vs ~890 MB nominal), `gzip -t` valida integridad, y una inspección frame a frame confirma ausencia del bloque de ruido NORDIC. Son truncamientos genuinos, no errores de descarga. El criterio del pipeline es excluir de los análisis de conectividad cualquier run con < 400 frames útiles (equivalente a < 12 min de rest), lo que implica descartar los dos runs de P6/ses-9 y el REST3 de P6/ses-5.

Para P7/ses-7, el bloque de 3 frames de ruido aparece al final de la serie (frames 514–516) en lugar de en la posición estándar, lo que es consistente con una variante de la secuencia NORDIC del CMRR. NORDIC los consume idénticamente en cualquiera de las dos posiciones.

4.0.4 4.4 T1w multi-echo (MEMPRAGE 4-eco) sólo en sub-P7

La inspección del directorio `anat/` reveló que **sub-P7 emplea una secuencia anatómica distinta** a la del resto de la cohorte. La diferencia se infiere de los JSON sidecars de `*_T1w.json`:

Table 7: Diferencias en la adquisición anatómica entre sub-P7 y el resto de ds006072 (extraídas de los JSON sidecars). El `ProtocolName` de P7 conserva la etiqueta `2e` aunque la secuencia adquiere 4 ecos individuales — una rareza de la nomenclatura del protocolo Siemens.

Parámetro	P1–P6	P7
<code>ProtocolName</code>	<code>t1_mpr_0p9mm</code>	<code>T1w_MPR_vNav_woNav_2e_0.5mm</code>
Tipo de secuencia	MPRAGE single-echo	MEMPRAGE 4-eco con vNav
Resolución nominal	0.9 mm isotrópica	0.5 mm isotrópica
<code>InstitutionName</code> / scanner	Washington_University / Prisma_fit	WUSTL_bay1 / Prisma
<code>ProcedureStepDescription</code>	<code>head_Nicol</code>	<code>HEAD_Laumann</code>
Sesión anatómica	ses-1	ses-0
Runs T1w por sesión	1–2	6 acquisitions multi-echo, pero el escáner sólo generó 1 RMS combine (de run-01)

Parámetro	P1–P6	P7
Echos por run	1	4 ecos individuales por acquisition + 1 RMS combine sólo en run-01

P7 fue adquirido en otro sitio del consorcio (Laumann/bay1) con una variante MEM-PRAGE multi-eco con navegación volumétrica vNav: el escáner registra los 4 ecos individuales y, además, entrega un volumen RMS-combinado (`SeriesDescription: "T1w_MPR_vNav_woNav_2e_0.5mm_RMS"`, `ImageType` con marcador `MEAN`). La conversión BIDS de origen distribuye ambos productos en `anat/`:

```
sub-P7_ses-0_run-NN_T1w.nii.gz          ← RMS combine (1 por run, ImageType MEAN)
sub-P7_ses-0_run-NN_echo-{1,2,3,4}_T1w.nii.gz ← ecos individuales (4 por run)
```

es decir, **24 ecos individuales + 1 volumen RMS combine = 25 archivos *_T1w.nii.gz** en la sesión anatómica de P7, frente a 1–2 archivos T1w en el resto de la cohorte. El escáner sólo generó RMS combine para la primera acquisition (`run-01`); las acquisitions `run-02` a `run-06` aparecen sólo como ecos individuales sin combinado. PyBIDS (y por tanto fMRIPrep) interpreta los 25 archivos *_T1w.nii.gz como inputs equivalentes para la construcción del template anatómico (workflow `anat_template_wf`), e invoca `mri_robust_template` de FreeSurfer (Reuter et al., 2010) con los 25 volúmenes como *time points* a registrar entre sí. Como los ecos individuales del MEMPRAGE tienen contraste T1w marcadamente distinto entre sí (TE 1.81 ms para `echo-1` vs TEs progresivamente mayores), 21 de las 24 registraciones inter-eco no convergen y FreeSurfer termina con `Segmentation fault (core dumped)`, abortando el `anat_fit_wf` y bloqueando todo el procesamiento posterior de P7 (incidente reproducido el 02/may/2026).

A nivel de denoising NORDIC esta heterogeneidad **no requiere acción**: NORDIC se aplica sólo a las series temporales BOLD (5 ecos $\times$ ~510 frames; trazabilidad en los campos `DenoisingMethod`, `NORDICMode: "complex"`, `NoiseVolumesUsed`, etc. del JSON sidecar BOLD de P7). Los T1w no pasan por NORDIC en ninguno de los sujetos del dataset, por lo que la entidad `rec-NDNORM` que llevan los T1w de P1–P6 funciona como marcador de sesión procesada y no de imagen denoised.

La solución estándar es **descartar los ecos individuales** del directorio `anat/` antes de invocar fMRIPrep, conservando únicamente el RMS combine generado por el escáner — que es exactamente la imagen que fMRIPrep espera para una secuencia MPRAGE convencional. La operación se ejecuta una vez como pre-flight de fMRIPrep en §5.4.1. Las cuatro restantes peculiaridades de P7 (sitio distinto, resolución T1w 0.5 vs 0.9 mm, sesión `ses-0` vs `ses-1`, número de runs anatómicos) no afectan al funcionamiento de fMRIPrep: la normalización a MNI152NLin2009cAsym homologa la resolución y el mapping `ses-{drug1, drug2}` se respeta desde `participants.tsv` con independencia del label de la sesión anatómica.

5 5. Correcciones del dataset ds006072

5.1 5.1 Creación de participants.tsv

El dataset ds006072 no incluye un archivo `participants.tsv` en la raíz. Aunque fMRIPrep no lo requiere (descubre los sujetos automáticamente por la estructura de directorios BIDS), este archivo es recomendado por el estándar BIDS y resulta útil para el análisis estadístico posterior, ya que centraliza las variables de grupo necesarias para los contrastes PSIL vs. MTP.

El README del dataset proporciona únicamente estadísticas demográficas agregadas (edad media 34.1 ± 9.8 años, 3 mujeres, 6 caucásicos), sin desglose individual. No obstante, la información del orden de administración de cada sujeto sí está disponible, y es la variable más relevante para nuestro análisis de conectividad funcional. El siguiente código genera el archivo:

```
import csv
import os

tsv_path = "F:/JIVillL/openneuro/ds006072/participants.tsv"

# Mapeo extraído de session_data.csv y README
subjects = [
    {"participant_id": "sub-P1", "drug1": "MTP", "drug2": "PSIL", "ses_drug1": 6, "ses_drug2": 6},
    {"participant_id": "sub-P2", "drug1": "PSIL", "drug2": "MTP", "ses_drug1": 10, "ses_drug2": 10},
    {"participant_id": "sub-P3", "drug1": "MTP", "drug2": "PSIL", "ses_drug1": 8, "ses_drug2": 8},
    {"participant_id": "sub-P4", "drug1": "MTP", "drug2": "PSIL", "ses_drug1": 6, "ses_drug2": 6},
    {"participant_id": "sub-P5", "drug1": "PSIL", "drug2": "MTP", "ses_drug1": 6, "ses_drug2": 6},
    {"participant_id": "sub-P6", "drug1": "MTP", "drug2": "PSIL", "ses_drug1": 5, "ses_drug2": 5},
    {"participant_id": "sub-P7", "drug1": "PSIL", "drug2": "MTP", "ses_drug1": 7, "ses_drug2": 7}
]

with open(tsv_path, "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=subjects[0].keys(), delimiter="\t")
    writer.writeheader()
    writer.writerows(subjects)

print(f"Creado: {tsv_path}")
```

El archivo resultante tendrá la siguiente estructura:

participant_id	drug1	drug2	ses_drug1	ses_drug2
sub-P1	MTP	PSIL	6	11
sub-P2	PSIL	MTP	10	16
sub-P3	MTP	PSIL	8	12
sub-P4	MTP	PSIL	6	10
sub-P5	PSIL	MTP	6	10
sub-P6	MTP	PSIL	5	9
sub-P7	PSIL	MTP	7	11

Este archivo permite identificar directamente qué sesión corresponde a psilocibina y cuál a metilfenidato para cada sujeto, facilitando la construcción de los contrastes PSIL vs. MTP en el análisis de conectividad funcional. `participants.tsv` lista los siete sujetos del dataset por completitud BIDS; el análisis, sin embargo, excluye a sub-P2 (§1.2), de modo que el N de los contrastes PSIL vs. MTP es 6.

5.2 Corrección de metadatos BIDS: campo TaskName

5.2.1 Descripción del problema

La especificación BIDS (Gorgolewski et al., 2016) establece que el campo `TaskName` en los archivos JSON sidecar debe coincidir con la etiqueta `task-<label>` del nombre de archivo correspondiente. En ds006072, los archivos funcionales están correctamente nombrados con etiquetas BIDS limpias (`task-BOLDREST1`, `task-BOLDREST2`, ..., `task-BOLDREST6` en P4/P6/P7; ver §4), pero el campo `TaskName` dentro de los JSON sidecars conserva el nombre del protocolo DICOM original del escáner Siemens Prisma:

Etiqueta en filename	TaskName en JSON (incorrecto)	TaskName correcto
<code>task-BOLDREST1</code>	<code>BOLDREST12mmmb6me5ipat2ppf68NORDICME52mm</code>	<code>BOLDREST1</code>
<code>task-BOLDREST1</code>	<code>BOLDREST1NORDICME52mm</code>	<code>BOLDREST1</code>
<code>task-BOLDREST2</code>	<code>BOLDREST22mmmb6me5ipat2ppf68NORDICME52mm</code>	<code>BOLDREST2</code>
<code>task-BOLDREST2</code>	<code>BOLDREST2NORDICME52mm</code>	<code>BOLDREST2</code>
<code>task-BOLDREST{3..6}</code>	<code>BOLDREST{3..6}NORDICME52mm</code>	<code>BOLDREST{3..6}</code>

El regex de corrección descrito más abajo captura todos los `BOLDRESTk` independientemente del número (no está hardcodeado a 1/2), por lo que opera uniformemente sobre cualquier número de tareas.

5.2.2 5.2.2 Causa de la discrepancia

El dataset fue convertido de DICOM a NIfTI/BIDS mediante **niix2bids v2.5.0** (indicado en el README: `GeneratedBy: niix2bids==v2.5.0`) y **dcm2niix v1.0.20240202** (indicado en los JSON sidecars: `"ConversionSoftware": "dcm2niix"`). Durante la conversión, `dcm2niix` genera el campo `TaskName` a partir del campo DICOM `ProtocolName/SeriesDescription` (en este caso, el nombre completo del protocolo de la secuencia CMRR multi-echo: `BOLD_REST1_2mm_mb6_me5_ipat2_ppf68_FOV220_1kblck_on`). Posteriormente, `niix2bids` renombró los archivos a etiquetas BIDS conformes (`task-BOLDREST1`, `task-BOLDREST2`) pero **no actualizó** el campo `TaskName` en los JSON sidecars para que coincidiera con las nuevas etiquetas.

Los dos valores distintos de `TaskName` por tarea (`BOLDREST12mmmb6me5ipat2ppf68FOV2201kblckon` y `BOLDREST1NORDICME52mm`) reflejan que Siegel et al. (2024) emplearon dos variantes del protocolo de adquisición: una estándar y otra compatible con **NORDIC PCA** (*Noise Reduction with DIstribution Corrected PCA*; Moeller et al., 2021; Vizioli et al., 2021), que incluye 3 volúmenes de ruido al final de cada run para estimar el nivel de ruido térmico.

5.2.3 5.2.3 Justificación de la corrección

1. **Cumplimiento BIDS:** la discrepancia constituye un error de validación BIDS (*"Invalid JSON file. The file is not formatted according to the schema"*), como reporta el validador de OpenNeuro para este dataset. La especificación BIDS requiere que `TaskName` coincida con el label del filename (Gorgolewski et al., 2016, §4.1).
2. **Compatibilidad con herramientas downstream:** aunque `fMRIPrep` utiliza `PyBIDS` para el descubrimiento de archivos (basándose en las entidades del filename, no en el JSON `TaskName`), otras herramientas del ecosistema `NiPreps` y de análisis (`fMRIPost-AROMA`, `fitlins`, `Nilearn`) pueden utilizar el campo `TaskName` del JSON sidecar para filtrar, agrupar o etiquetar los datos. Una discrepancia puede provocar errores silenciosos o agrupaciones incorrectas.
3. **Reproducibilidad:** un dataset BIDS-válido garantiza que cualquier pipeline BIDS-compatible procesará los datos correctamente sin intervención manual.

5.2.4 5.2.4 Corrección automática

El siguiente script corrige el campo `TaskName` en todos los JSON sidecars BOLD del dataset, extrayendo la etiqueta correcta directamente del nombre de archivo según la convención BIDS (`task-<label>`). La corrección opera mediante sustitución directa sobre el texto JSON (expresión regular), sin deserializar y reserializar el archivo completo. Esto preserva exactamente la indentación original (2 o 4 espacios según el archivo), los saltos de línea Unix (`\n`), la precisión

numérica de todos los campos (tiempos de eco, *slice timing*, etc.) y el `\n` final. Solo se modifica el valor de `TaskName`:

```
#####
# Corrección de metadatos BIDS: campo TaskName
# Los JSON sidecars de ds006072 conservan el ProtocolName DICOM original
# en lugar del label BIDS correcto. Corregimos para cumplir BIDS v1.6.0.
#
# Estrategia: sustitución regex sobre el texto crudo del JSON para
# preservar formato, indentación, line endings y precisión numérica.
#
# Referencia: Gorgolewski et al. (2016), Scientific Data 3:160044
#             BIDS Specification v1.6.0, §4.1 (Task fMRI)
#####

import os
import re
import json
import glob

BIDS_DIR = "F:/JIVillL/openneuro/ds006072"
# Buscar todos los JSON sidecars BOLD
bold_jsons = glob.glob(
    os.path.join(BIDS_DIR, "sub-*/ses-*/func/*_bold.json")
)

fixed = 0
skipped = 0
errors = []

# Regex para extraer TaskName del texto JSON sin deserializar
# Captura: "TaskName": "valor_actual"
TASKNAME_RE = re.compile(r'"TaskName"\s*:\s*"([~"]*)"')

for json_path in sorted(bold_jsons):
    fname = os.path.basename(json_path)

    # Extraer el label de tarea del filename (task-<label>)
    match = re.search(r"task-([A-Za-z0-9]+)", fname)
    if not match:
        print(f" Sin task label: {fname}")
        continue
```

```

correct_taskname = match.group(1) # e.g., "BOLDREST1"

# Leer el archivo como bytes para preservar line endings exactos
with open(json_path, "rb") as f:
    raw = f.read()

text = raw.decode("utf-8")

# Extraer TaskName actual del texto
tn_match = TASKNAME_RE.search(text)
if not tn_match:
    errors.append(f" SIN TaskName: {fname}")
    continue

current_taskname = tn_match.group(2)

if current_taskname == correct_taskname:
    skipped += 1
    continue

# Sustituir solo el valor de TaskName, preservando todo lo demás
new_text = TASKNAME_RE.sub(rf'\1"{correct_taskname}"', text, count=1)

# Verificar que solo cambió lo esperado
if text.count('"TaskName"') != 1:
    errors.append(f" MULTIPLE TaskName: {fname}")
    continue

if new_text == text:
    errors.append(f" SIN CAMBIO INESPERADO: {fname}")
    continue

# Escribir como bytes para preservar line endings (\n, no \r\n)
with open(json_path, "wb") as f:
    f.write(new_text.encode("utf-8"))

# Verificar que el JSON resultante es válido
try:
    check = json.loads(new_text)
    assert check["TaskName"] == correct_taskname
except (json.JSONDecodeError, AssertionError) as e:
    errors.append(f" JSON INVALIDO tras correccion: {fname} ({e})")

```

```

        # Restaurar original
        with open(json_path, "wb") as f:
            f.write(raw)
        continue

    print(f" {fname}: '{current_taskname}' -> '{correct_taskname}'")
    fixed += 1

print(f"\nCorregidos: {fixed} archivos")
print(f"Ya correctos: {skipped} archivos")
print(f"Total procesados: {fixed + skipped} archivos")

if errors:
    print(f"\nERRORES ({len(errors)}):")
    for e in errors:
        print(e)
else:
    print("Sin errores.")

```

Cuándo ejecutar

Si se aplica, ejecutar **después** de completar la descarga y **antes** de NORDIC y fMRIPrep. El glob `*_bold.json` captura tanto los sidecars de magnitud (`*_part-mag_bold.json`) como los de fase (`*_part-phase_bold.json`); ambos se corrigen en una sola ejecución, antes de que NORDIC consuma las imágenes complejas. No modifica los datos de imagen (NIFTI), solo los metadatos JSON. Es idempotente: puede re-ejecutarse sin efectos adversos.

5.3 5.3 Corrección de metadatos BIDS: campos B0FieldIdentifier/B0FieldSource

5.3.1 5.3.1 Descripción del problema

Los archivos JSON de los fieldmaps de ds006072 no contienen los campos `B0FieldIdentifier` ni `B0FieldSource`, ni el campo legacy `IntendedFor`. La documentación de fMRIPrep v25.2.5 (SDCFlows 2.0) establece dos mecanismos de vinculación fieldmap-BOLD:

1. `B0FieldIdentifier/B0FieldSource` (preferido, BIDS 1.5.0): el JSON del fieldmap declara un identificador (`B0FieldIdentifier`) y el JSON del BOLD lo referencia (`B0FieldSource`).

2. **IntendedFor** (legacy): el JSON del fieldmap lista explícitamente los archivos BOLD a los que aplica.

En ausencia de ambos campos, SDCFlows v2.x en fMRIPrep 25.2.5 **no realiza matching automático** para fieldmaps PEPOLAR (spin-echo AP/PA), sino que omite la corrección de distorsión silenciosamente.

5.3.2 Estrategia de corrección

Se añaden los campos `B0FieldIdentifier` y `B0FieldSource` según el estándar BIDS 1.5.0:

- **Fieldmaps** (fmap/*_epi.json): se añade "B0FieldIdentifier": ["pepolar_{sujeto}_{sesión}"] (p. ej. pepolar_sub-P1_ses-6). Todos los fieldmaps de una misma sesión (AP run-1, AP run-2, PA run-1, PA run-2) comparten el mismo identificador, indicando que forman un único conjunto PEPOLAR.
- **BOLD** (func/*_bold.json): se añade "B0FieldSource": ["pepolar_{sujeto}_{sesión}"], referenciando el identificador del par PEPOLAR de la misma sesión.

Esta vinculación explícita es el mecanismo preferido por SDCFlows y garantiza que cada serie BOLD se corrija con los fieldmaps de su propia sesión de adquisición, respetando la geometría de distorsión específica de cada sesión (posición de cabeza, shimming).

5.3.3 Corrección automática

```
#####
# Corrección de metadatos BIDS: B0FieldIdentifier / B0FieldSource
# Los JSON de ds006072 carecen de vinculación explícita fieldmap→BOLD,
# impidiendo que SDCFlows (fMRIPrep 21.0) aplique corrección de distorsión.
#
# Estrategia: inserción de campos B0Field* en el texto crudo del JSON
# para preservar formato, indentación, line endings y precisión numérica.
#
# Referencia: BIDS Specification v1.5.0+, §8.9.4 (Fieldmap metadata)
#             fMRIPrep Documentation v25.2.5, §4.2.4 (SDCFlows)
#####

import os
import re
import json
import glob
```

```

BIDS_DIR = "F:/JIVillL/openneuro/ds006072"

def set_json_field(json_path, field_name, field_value):
    """Añade o actualiza un campo en un JSON preservando formato.

    Usa sustitución regex sobre el texto crudo para mantener indentación,
    line endings y precisión numérica de todos los campos existentes.
    Retorna "added", "updated" o "unchanged" según la acción realizada.
    """
    with open(json_path, "rb") as f:
        raw = f.read()
        text = raw.decode("utf-8")

    # Detectar indentación del archivo (tab o espacios)
    indent_match = re.search(r'\n([\t]+)', text)
    indent = indent_match.group(1) if indent_match else "\t"
    value_str = json.dumps(field_value)

    if f'"{field_name}"' in text:
        # Campo ya existe → comprobar si el valor es el correcto
        existing = json.loads(text)
        if existing.get(field_name) == field_value:
            return "unchanged"
        # Reemplazar el valor existente (soporta arrays y strings)
        pattern = rf'("{field_name}"\s*:\s*)(\[[.*?\]|"[^"]*"')
        new_text = re.sub(pattern, rf'\g<1>{value_str}', text)
        action = "updated"
    else:
        # Campo no existe → insertar antes del cierre final
        new_field = f',\n{indent}"{field_name}": {value_str}'
        new_text = re.sub(r'\s*\}\s*$', f'{new_field}\n}\n', text)
        action = "added"

    # Verificar que el JSON resultante es válido
    try:
        parsed = json.loads(new_text)
        assert field_name in parsed
        assert parsed[field_name] == field_value
    except (json.JSONDecodeError, AssertionError) as e:
        print(f" ERROR JSON inválido: {os.path.basename(json_path)} ({e})")
        return "unchanged"

```

```

with open(json_path, "wb") as f:
    f.write(new_text.encode("utf-8"))
return action

# ---- Descubrir todas las sesiones con fieldmaps ----
fmap_dirs = sorted(glob.glob(os.path.join(BIDS_DIR, "sub-*/ses-*/fmap")))

added = 0
updated = 0
unchanged = 0
errors = []

for fmap_dir in fmap_dirs:
    ses_dir = os.path.dirname(fmap_dir)

    match = re.search(r"(sub-[/\\]+)([/\\])(ses-\d+)", fmap_dir)
    if not match:
        errors.append(f" No se pudo extraer sujeto/sesión de: {fmap_dir}")
        continue

    sub_label = match.group(1) # e.g. "sub-P1"
    ses_label = match.group(2) # e.g. "ses-6"
    identifier = f"pepolar_{sub_label}_{ses_label}"

    # ---- B0FieldIdentifier en fieldmaps ----
    fmap_jsons = sorted(glob.glob(os.path.join(fmap_dir, "*_epi.json")))
    for fj in fmap_jsons:
        result = set_json_field(fj, "B0FieldIdentifier", [identifier])
        if result == "added":
            print(f" + B0FieldIdentifier: {os.path.basename(fj)} → [{identifier}]")
            added += 1
        elif result == "updated":
            print(f" ~ B0FieldIdentifier: {os.path.basename(fj)} → [{identifier}] (actualizado)")
            updated += 1
        else:
            unchanged += 1

    # ---- B0FieldSource en BOLD de la misma sesión ----
    func_dir = os.path.join(ses_dir, "func")
    if not os.path.isdir(func_dir):
        continue

```

```

bold_jsons = sorted(glob.glob(os.path.join(func_dir, "*_bold.json")))
for bj in bold_jsons:
    result = set_json_field(bj, "B0FieldSource", [identifier])
    if result == "added":
        print(f" + B0FieldSource: {os.path.basename(bj)} → [{identifier}]")
        added += 1
    elif result == "updated":
        print(f" ~ B0FieldSource: {os.path.basename(bj)} → [{identifier}] (actualizado)")
        updated += 1
    else:
        unchanged += 1

print(f"\nArchivos añadidos: {added}")
print(f"Archivos actualizados: {updated}")
print(f"Sin cambios: {unchanged}")
print(f"Total modificados: {added + updated}")

if errors:
    print(f"\nERRORES ({len(errors)}):")
    for e in errors:
        print(e)
else:
    print("Sin errores.")

```

i Cuándo ejecutar

Ejecutar **después** de la corrección de TaskName y **antes** de NORDIC y fMRIPrep. El glob *_bold.json captura tanto los sidecars de magnitud (*_part-mag_bold.json) como los de fase (*_part-phase_bold.json), pero B0FieldSource solo es relevante para los archivos que entrarán a fMRIPrep (los part-mag denoised por NORDIC); los sidecars de fase se corrigen por consistencia BIDS aunque se descarten tras NORDIC. No modifica los datos de imagen (NIfTI), solo los metadatos JSON. Es idempotente: puede re-ejecutarse sin efectos adversos. Tras esta corrección, fMRIPrep deberá reportar "Stage 3: Adding fieldmap correction" en lugar de "No fieldmap correction - skipping Stage 3" para cada serie BOLD.

5.4 5.4 Correcciones para sub-P7

5.4.1 5.4.1 Descartar T1w multi-echo

Como se documenta en §4.4, el `anat/` de sub-P7 contiene los 4 ecos individuales del MEMPRAGE de cada una de las 6 acquisitions, además de un único RMS combine que el escáner generó para `run-01`. fMRIPrep interpreta los 25 archivos `*_T1w.nii.gz` como inputs equivalentes para `mri_robust_template` y termina con `Segmentation fault (core dumped)` en la fase `anat_template_wf`, abortando todo el procesamiento de P7. La solución es **eliminar los 24 ecos individuales del árbol BIDS** y conservar únicamente el RMS combine generado por el escáner (`sub-P7_ses-0_run-01_T1w.nii.gz`, 1 archivo), que es exactamente la imagen que fMRIPrep espera para una secuencia MPRAGE convencional.

```
# Pre-flight P7 - eliminar los T1w multi-echo individuales del MEMPRAGE del
# árbol BIDS fuente. Conserva sólo el RMS combine que el escáner entrega como
# `sub-P7_ses-0_run-01_T1w.nii.gz`. Backup íntegro de los 24 ecos disponible
# en disco secundario externo al proyecto.
$src = "F:\JIVilll\openneuro\ds006072\sub-P7\ses-0\anat"
if (Test-Path $src) {
    $files = Get-ChildItem $src -Filter "*_echo-*_T1w.*"
    if ($files.Count -gt 0) {
        $files | Remove-Item -Force
        Write-Host "Borrados $($files.Count) ecos T1w individuales en $src"
    } else {
        Write-Host "Nada que borrar (anat/ ya limpio)"
    }
} else {
    Write-Host "ERROR: no existe $src"
}
```

Verificación posterior — el `anat/` debe contener un único `*_T1w.nii.gz` (el RMS combine de `run-01`) y los seis `*_T2w.nii.gz` (T2-SPACE separado del MEMPRAGE, que sí está presente en las 6 acquisitions y no requiere intervención en este pre-flight):

```
Get-ChildItem F:\JIVilll\openneuro\ds006072\sub-P7\ses-0\anat\*_T1w.nii.gz | Measure-Object
# Count debe ser 1 (sólo el RMS combine de run-01, sin ecos)
Get-ChildItem F:\JIVilll\openneuro\ds006072\sub-P7\ses-0\anat\*_T2w.nii.gz | Measure-Object
# Count debe ser 6 (T2w-SPACE de run-01..run-06, intactos)
```

⚠ Segundo pre-flight encadenado en sub-P7: homogeneizar resolución T1w

Este pre-flight resuelve el crash de `mri_robust_template`, pero P7 requiere **además** una segunda intervención antes de invocar fMRIPrep: resamplear el T1w RMS combine (0.5 mm) y los 6 T2w (0.8 mm) a 0.9 mm para igualar la resolución de adquisición del resto de la cohorte. Sin este paso, `recon-all` de FreeSurfer sobre el MEMPRAGE 0.5 mm excede las 48 h de cómputo (incidente reproducido el 03-05/may/2026) e introduce además una asimetría metodológica innecesaria. La justificación, el script de downsample y la verificación posterior se documentan en §5.4.2. **Ambos pre-flights deben ejecutarse en orden** (delección de ecos → downsample) — el segundo asume que el primero ya ha limpiado el `anat/`.

5.4.2 Homogeneizar resolución T1w a 0.9 mm

Tras eliminar los ecos individuales (§5.4.1), sub-P7 conserva en `anat/`:

- **1 archivo T1w RMS combine** (`sub-P7_ses-0_run-01_T1w.nii.gz`) adquirido a **0.5 mm isotrópica** con secuencia MEMPRAGE 4-eco con vNav (§4.4).
- **6 archivos T2w** (`sub-P7_ses-0_run-{01..06}_T2w.nii.gz`) adquiridos con secuencia T2-SPACE separada a **0.8 mm isotrópica**, uno por cada acquisition repetida.

Frente a estas dos resoluciones, el resto de la cohorte (P1–P6) lleva tanto T1w como T2w a **0.9 mm isotrópica** (`acq-t1mpr0p9mm` y `acq-t2spc0p9mmiso`). Esta heterogeneidad de resolución provoca dos problemas concretos:

1. **recon-all no escala benignamente con la resolución.** El número de vértices corticales crece como $(0.9/0.5)^3 \approx 5.8\times$, y las fases dominantes (`mris_make_surfaces`, `mris_topology_fixer`) iteran sobre ese conteo. Una ejecución empírica del 03-05/may/2026 sobre el i5-10400F + 24 GB WSL del entorno (con `--omp-nthreads 2`) **superó las 48 h** sin completar `_autorecon_surfs1` (hemisferio derecho), comportamiento desproporcionado incluso para 0.5 mm. Causa probable adicional: el contraste WM/GM atenuado del RMS-combine de 4 ecos (promediado de TEs 1.81–7.27 ms) provoca iteración patológica del topology fixer sobre superficies de ~400–500k vértices.
2. **Heterogeneidad de resolución entre P7 y el resto de la cohorte.** Como el análisis de conectividad funcional (PsyConn) opera **íntegramente en espacio volumétrico MNI152NLin2009cAsym** con BOLD nativo a 2 mm y atlas Schaefer 200 + Tian S2 (parcelas de ~200–250 vóxeles cada una a 2 mm), **la resolución anatómica nativa de 0.5 mm de P7 no se traduce en ninguna ganancia downstream**: el cuello de botella de granularidad es la resolución BOLD (2 mm) y, por encima de ella, la agregación parcelar. Resamplear los T1w de P7 de 0.5 a 0.9 mm antes de fMRIPrep sacrifica detalle

anatómico fino que el pipeline ya iba a perder de todos modos en el resamplado a MNI, a cambio de dos beneficios:

- **recon-all** se ejecuta en el rango de tiempo nominal de la cohorte (~5 h, idéntico al de P1–P6).
- El comando de fMRIPrep para sub-P7 es **idéntico** al del resto de la cohorte: mismo brain extraction, misma segmentación, mismo **bbregister** para corregistro BOLD T1w, mismas superficies a la misma resolución. Cualquier asimetría metodológica entre P7 y P1–P6 desaparece.

El material original a 0.5 mm (T1w) y 0.8 mm (T2w) está respaldado en disco secundario externo al proyecto, de modo que la operación es reversible si en el futuro fuera necesario re-procesar P7 a resolución nativa (p.ej. para métricas surface-based de alta resolución). El script lee con **nibabel**, resamplado con **nilearn.image.resample_img** (interpolación lineal — preserva contraste tisular sin introducir aliasing como el nearest-neighbor) y, si fuera necesario, recorta el FOV antes de sobrescribir in-place. Es idempotente: la segunda ejecución detecta zooms ya = (0.9, 0.9, 0.9) y FOV 256 mm en todas las dimensiones, e imprime SKIP sin escribir.

Cuatro detalles importantes del script:

1. El glob **run-??_T1w.nii.gz** (dos dígitos exactos antes del sufijo) excluye explícitamente cualquier ***_echo-*_T1w.*** que pudiera haber sobrevivido a §5.4.1.
2. Cubre **T1w y T2w** en una misma pasada (mismo target 0.9 mm), porque la resolución T2-SPACE de P7 (0.8 mm) tampoco coincide con la de la cohorte (0.9 mm) y el desajuste introduciría resamplados internos en **recon-all -T2**.
3. El **sub-P7_ses-0_run-05_T2w.nii.gz** viene de la conversión BIDS con un eje temporal espurio de longitud 1 (**shape = (208, 300, 320, 1)**); el script lo **squeeze**-a a 3D antes de resamplado para evitar que fMRIPrep lo trate como serie temporal.
4. **Recorte de FOV 256 mm**: FreeSurfer en modo **-hires** (que fMRIPrep activa automáticamente al detectar voxel < 1 mm) tiene un **límite duro** de 256 mm de FOV en cada dimensión. Tras downsamplado, el T1w queda con **shape (161, 268, 285)** a 0.9 mm, lo que da FOV (145, 241, **256.5**) mm — sólo 0.5 mm de exceso en z. La conformación interna **mri_convert --conform_min** levanta esto a un cubo 285³ y la validación posterior aborta con **ERROR! FOV=256.500 > 256** (incidente reproducido el 05/may/2026 a las 07:19 UTC). El script recorta simétricamente cualquier eje con FOV > 256 mm y actualiza el **affine** consecuentemente para preservar la geometría espacial. Para el T1w esto supone retirar 1 slice en z (285→284, FOV z 256.5→255.6 mm). El bloque del T2w probablemente no requiera recorte (su FOV es ~256 mm justos tras el downsamplado), pero la función lo verifica por si acaso.

```
import nibabel as nib
from nilearn.image import resample_img
from pathlib import Path
```

```

import numpy as np

TARGETS = [Path("F:/JIVillL/openneuro/ds006072/sub-P7/ses-0/anat")]
TARGET_ZOOM = (0.9, 0.9, 0.9)
MAX_FOV_MM = 256.0 # límite duro de FreeSurfer en modo -hires
TOL = 1e-3

def squeeze_trivial_axes(img):
    """Quita ejes 4D de longitud 1. Caso conocido: sub-P7_ses-0_run-05_T2w."""
    if img.ndim == 4 and img.shape[3] == 1:
        data = np.asarray(img.dataobj).squeeze(axis=3)
        return nib.Nifti1Image(data, img.affine, img.header), True
    return img, False

def already_at_target_zoom(img, target=TARGET_ZOOM, tol=TOL):
    return all(abs(o - n) < tol for o, n in zip(img.header.get_zooms()[3], target))

def fov_exceeds(img, max_fov=MAX_FOV_MM):
    return any(s * z > max_fov for s, z in zip(img.shape[3], img.header.get_zooms()[3]))

def downsample_to(img, target_zoom=TARGET_ZOOM):
    old = img.header.get_zooms()[3]
    new_affine = img.affine.copy()
    for i in range(3):
        new_affine[3, i] *= target_zoom[i] / old[i]
    # force_resample=True y copy_header=True son los defaults a partir de
    # Nilearn 0.13.0; se fijan explícitos para silenciar FutureWarning.
    return resample_img(img, target_affine=new_affine, interpolation="linear",
                        force_resample=True, copy_header=True)

def crop_to_max_fov(img, max_fov=MAX_FOV_MM):
    """Recorta simétricamente cualquier eje con FOV > max_fov y actualiza affine."""
    data = np.asarray(img.dataobj)
    zooms = img.header.get_zooms()[3]
    shape = list(data.shape[3])
    crops = [(0, 0)] * 3
    for axis in range(3):

```

```

        fov = shape[axis] * zooms[axis]
        if fov > max_fov:
            target_n = int(np.floor(max_fov / zooms[axis]))
            remove = shape[axis] - target_n
            crops[axis] = (remove // 2, remove - remove // 2)
    if all(c == (0, 0) for c in crops):
        return img
    sl = tuple(slice(c[0], shape[a] - c[1]) for a, c in enumerate(crops))
    if data.ndim == 3:
        data_c = data[sl]
    else:
        data_c = data[sl + (slice(None),) * (data.ndim - 3)]
    # Si recortamos desde el inicio de un eje, el origen mundial se desplaza.
    offset_vox = np.array([c[0] for c in crops])
    origin_shift = img.affine[:3, :3] @ offset_vox
    new_affine = img.affine.copy()
    new_affine[:3, 3] += origin_shift
    return nib.Nifti1Image(data_c, new_affine, img.header)

def fix_inplace(src):
    """Ajusta un volumen anatómico para fMRIPrep + FreeSurfer hires.
    Aplica en orden: squeeze 4D->3D, downsample a 0.9 mm, crop a FOV<=256 mm."""
    img = nib.load(src)
    actions = []

    img, did_squeeze = squeeze_trivial_axes(img)
    if did_squeeze:
        actions.append("squeeze 4D->3D")

    if not already_at_target_zoom(img):
        old_zooms = tuple(round(x, 3) for x in img.header.get_zooms()[:3])
        img = downsample_to(img)
        new_zooms = tuple(round(x, 3) for x in img.header.get_zooms()[:3])
        actions.append(f"resample {old_zooms}->{new_zooms}")

    if fov_exceeds(img):
        old_shape = img.shape[:3]
        img = crop_to_max_fov(img)
        if img.shape[:3] != old_shape:
            actions.append(f"crop {old_shape}->{img.shape[:3]}")

```

```

if actions:
    nib.save(img, src)
    print(f"OK {src.name}: {' | '.join(actions)}")
else:
    print(f"SKIP {src.name}: ya cumple zoom={TARGET_ZOOM} y FOV<={MAX_FOV_MM} mm")

for anat_dir in TARGETS:
    if not anat_dir.exists():
        continue
    print(f"\n=== {anat_dir.parent.parent.parent.name} / {anat_dir.parent.parent.name} ===")
    # Cubre T1w (sólo RMS combine, glob estricto excluye echo-*) y T2w.
    for src in sorted(anat_dir.glob("sub-P7_ses-0_run-??_T?w.nii.gz")):
        fix_inplace(src)

```

Verificación posterior — todos los *_T1w.nii.gz y *_T2w.nii.gz de sub-P7/ses-0/anat deben tener zooms (0.9, 0.9, 0.9), ser estrictamente 3D y tener FOV 256 mm en todas las dimensiones:

```

import nibabel as nib
from pathlib import Path

d = Path("F:/JIVillL/openneuro/ds006072/sub-P7/ses-0/anat")
files = sorted(d.glob("sub-P7_ses-0_run-??_T?w.nii.gz"))
print(f"{len(files)} archivos anatómicos en {d}")
for f in files:
    img = nib.load(f)
    zooms = tuple(round(x, 3) for x in img.header.get_zooms()[:3])
    shape = img.shape
    fov = tuple(round(s * z, 2) for s, z in zip(shape[:3], zooms))
    flags = []
    if zooms != (0.9, 0.9, 0.9): flags.append("ZOOM")
    if img.ndim != 3: flags.append("NDIM")
    if any(v > 256.0 for v in fov): flags.append("FOV")
    status = " ".join(flags) if flags else "OK"
    print(f" {f.name}: zooms={zooms}, shape={shape}, FOV={fov} mm [{status}]")
# Esperado: todos los archivos con [OK]. Cualquier flag (ZOOM/NDIM/FOV)
# indica que algún paso del fix_inplace no se aplicó.
# La anatomía de sub-P7 reside íntegramente en ses-0/anat (sesión única):
# 1 T1w + 6 T2w = 7 archivos, todos con zooms (0.9, 0.9, 0.9) y ndim == 3.

```

5.4.3 Independencia de NORDIC

El downsample y NORDIC son ramas **independientes** del DAG de procesamiento: NORDIC opera exclusivamente sobre series temporales BOLD (*_part-mag_bold.nii.gz y *_part-phase_bold.nii.gz) y nunca toca el T1w (los sidecars de los *_T1w.json no contienen los campos DenoisingMethod/NORDICMode/NoiseVolumesUsed que sí trazan el procesamiento NORDIC en los sidecars BOLD). El downsample puede ejecutarse en cualquier momento posterior a la conversión BIDS y anterior a fMRIPrep; en este pipeline se ejecuta **antes de NORDIC** por mera comodidad organizativa.

6 6. Estrategia incremental de procesamiento: Tier 0

El techo de cómputo por invocación de fMRIPrep en este TFM es el equivalente a **un sujeto P1-sized** (8 runs $\times$ 5 ecos = 40 run-ecos procesables en una única ejecución). Ese techo no cubre la totalidad del dataset en una sola corrida por sujeto — **sub-P5** acumula 16 runs, **sub-P7** 9, **sub-P3** y **sub-P1** 8 cada uno — y, sumado a los costes encadenados de NORDIC (~5–10 min/run-eco) y TEDANA (~15–30 min/run), el procesamiento completo del dataset abarcaría semanas de *wallclock*.

La solución adoptada es un **procesamiento incremental por tiers** alineado con la literatura de *precision functional mapping* (Laumann 2015, Gordon 2017, Gratton 2018) y con el propio diseño de ds006072 (series MSC-style por sujeto):

1. **Tier 0** — procesar el **mínimo de runs por sujeto y sesión** que permite *una primera vuelta end-to-end* del pipeline completo (NORDIC $\rightarrow$ fMRIPrep $\rightarrow$ TEDANA $\rightarrow$ parcelación $\rightarrow$ FC $\rightarrow$ Δ FC). Garantiza cobertura transversal de los 6 sujetos y valida ambas clases NORDIC (magnitude-only y complex) con N pequeño y rápido.
2. **Tiers +1, +2, ...** — **añadir los runs de reserva progresivamente** a medida que el proyecto avanza (tras validación del pipeline, en revisión de resultados intermedios), y **re-ejecutar el downstream** (TEDANA $\rightarrow$ FC) con los datos acumulados. NORDIC y fMRIPrep son per-run independientes: la anatomía del sujeto queda cacheada en **derivatives/sub-PX/anat/** y se reutiliza, y cada run nuevo pasa por NORDIC + fMRIPrep + TEDANA sin tocar lo ya procesado (el flujo incremental se desarrollará al escalar de tier).

Regla de selección Tier 0: *un run por sesión, BOLDREST1 run-1 por defecto*. Criterios de desempate (cuando BOLDREST1 no es viable): descartar los runs marcados **EXCLUDED** en el CSV de §3 y tomar el siguiente BOLDRESTn_run-1 íntegro disponible en la sesión. La elección de BOLDREST1_run-1 responde a dos criterios simultáneos: (i) **homogeneidad entre sujetos** — existe en las 12 sesiones de los 6 sujetos; (ii) **primer run de cada sesión**, típicamente con menos deriva/fatiga que los posteriores.

6.1 6.1 Runs excluidos permanentemente

Estos runs **no** se procesarán en ningún tier — la exclusión es estructural, no de planificación. El pre-flight de §3 los marca automáticamente como **EXCLUDED** por el criterio paramétrico $Q < Q_USABLE_MIN = 400$ (insuficiente para FC estable tras *censoring*); el chunk NORDIC los lista además en su **EXCLUDED_RUNS** local como cinturón de seguridad redundante:

Sujeto	Sesión	Task	Q observado	Q nominal	Motivo
sub-P6	ses-9	BOLDREST1	208	513	Run abortado prematuramente; insuficiente para FC estable tras <i>censoring</i> de movimiento.
sub-P6	ses-9	BOLDREST2	135	513	Run abortado prematuramente; descartado por el mismo criterio.
sub-P6	ses-5	BOLDREST3	245	513	Run corto (~50 % de longitud nominal); descartado por el mismo criterio.

Exclusiones = **3 runs** $\times$ **5 ecos** = **15 run-ecos** (5.6 % del volumen total). El fundamento empírico y el test de aplicabilidad técnica de NORDIC para estos runs está en §3.

6.2 6.2 Selección Tier 0 — primer pase del pipeline

$12 \text{ runs} \times 5 \text{ ecos} = \mathbf{60 \text{ run-ecos}}$ (~22 % del dataset de análisis, N=6). Cada sujeto aporta 2 sesiones $\times$ 1 run = 10 run-ecos, por debajo del techo P1 y suficiente para **fMRIPrep** en una

única invocación por sujeto.

#	Sujeto	Sesión	Task	Run	Modo NORDIC	Dir	Q esp.	Notas
1	sub-P1	ses-6	BOLDREST1	run-1	magnitude-only	AP	513	—
2	sub-P1	ses-11	BOLDREST1	run-1	magnitude-only	AP	513	—
3	sub-P3	ses-8	BOLDREST1	run-1	magnitude-only	AP	513	—
4	sub-P3	ses-12	BOLDREST1	run-1	magnitude-only	AP	513	—
5	sub-P4	ses-6	BOLDREST1	run-1	complex	PA	513	—
6	sub-P4	ses-10	BOLDREST1	run-1	complex	PA	513	—
7	sub-P5	ses-6	BOLDREST2	run-1	complex	PA	513	BOLDREST1 run-1 excluido en QC posterior
8	sub-P5	ses-10	BOLDREST2	run-1	complex	PA	513	BOLDREST1 run-1 excluido en QC posterior
9	sub-P6	ses-5	BOLDREST1	run-1	complex	PA	513	—
10	sub-P6	ses-9	BOLDREST2	run-1	complex	PA	513	BOLDREST1/2 excluidos (pre-flight)
11	sub-P7	ses-7	BOLDREST2	run-1	complex	PA	516	513 + 3 noise volumes terminales
12	sub-P7	ses-11	BOLDREST2	run-1	complex	PA	513	—

Cobertura por clase: **4 runs magnitude-only** (sub-P1, sub-P3) + **8 runs complex** (sub-P4–P7). Ambas clases con fase presente/ausente quedan validadas en Tier 0 antes de gastar cómputo en los tiers posteriores.

6.3 6.3 Runs en reserva — tiers posteriores

39 runs $\times$ 5 ecos = **195 run-ecos** (72 % del dataset de análisis), a distribuir en tiers futuros:

Sujeto	Runs reserva	Runs Tier 0	Total sujeto	Estrategia de ampliación
sub-P1	6	2	8	añadir run-2 de cada task + BOLDREST2 de ambas sesiones
sub-P3	6	2	8	añadir run-2 de cada task + BOLDREST2 de ambas sesiones
sub-P4	3	2	5	añadir BOLDREST2 (ambas sesiones) + BOLDREST3 (ses-6)
sub-P5	14	2	16	ampliar por bloques: BOLDREST2 run-1 (ambas ses) $\rightarrow$ BOLDREST3/4 $\rightarrow$ run-2
sub-P6	3	2	5	añadir BOLDREST2 y BOLDREST4 (ses-5) + BOLDREST4 (ses-9); BOLDREST1/2 ses-9 y BOLDREST3 ses-5 permanecen EXCLUDED
sub-P7	7	2	9	añadir BOLDREST2–4 (ses-7) + BOLDREST2–3 (ses-11)

Prioridad sugerida para Tier +1: duplicar minutos BOLD por sesión añadiendo **BOLDREST2 run-1** en todos los sujetos donde exista (P3 $\times$ 2, P4 $\times$ 2, P5 $\times$ 2, P6 ses-5, P7 $\times$ 2) = $\sim$ 10 runs adicionales, cruzando el umbral Laumann de 25–30 min/condición en la mayoría de sujetos.

6.4 6.4 Recuento global y trade-off metodológico

Bloque	Runs	Run-ecos	% dataset
Tier 0 (primer pase)	12	60	22 %
Reserva (tiers +1...)	39	195	72 %
Excluidos permanentes	3	15	6 %
Total dataset (N=6)	54	270	100 %

Con 1 run (~15 min) por sesión, las matrices FC individuales en Tier 0 quedan *por debajo* del umbral de estabilidad individual reportado por Laumann 2015 (25–30 min/condición). Se asume el ruido intra-sujeto como coste de tener **cobertura N=6 desde el primer pase**: los efectos ΔFC (PSIL vs. sesión basal) que sean replicables entre sujetos sobreviven al ruido individual, y los tiers sucesivos migran progresivamente cada sujeto hacia el régimen *precision mapping* conforme se añaden runs. La memoria del TFM puede reportar dos análisis separados: (a) inferencia grupal sobre el Tier 0 completo, (b) mapping individual detallado en los sujetos que alcancen régimen *precision* con los tiers posteriores.

6.5 6.5 Operacionalización: dataset filtrado por tier en prep/nordic/data/

Se adopta una estrategia limpia y reproducible: **generar una copia filtrada del BIDS con sólo los archivos del tier activo en derivatives/ds006072/prep/nordic/data/<tier>**. El chunk NORDIC apunta a esa subcarpeta del tier en lugar del dataset fuente completo en F:\JIVillL\openneuro\ds006072, y procesa *todo* lo que encuentre — la selección queda codificada implícitamente en la estructura de directorios, no en un filtro Python. NORDIC sobrescribe esas copias in-place, de modo que `prep/nordic/data/<tier>` pasa a ser directamente el output denoised que consultan el QC de PsiloPrep §7.9 y las etapas siguientes (fMRIPrep/TEDANA).

Ventajas:

1. **Selección auto-documentada**: el contenido de `prep/nordic/data/<tier>` es la definición del tier actual.
2. **Compatibilidad fMRIPrep**: fMRIPrep opera sobre un root BIDS; no hay *flag* nativo para restringir a un subconjunto de runs con la granularidad que necesitamos. Un BIDS filtrado evita `--bids-filter-file` manual y riesgos de inconsistencia entre etapas.
3. **Protege el dataset fuente**: NORDIC sobrescribe los `_part-mag_bold.nii.gz` *in place* (PsiloPrep §7). Trabajar sobre la copia en `prep/nordic/data/<tier>` deja intacto el dataset fuente pristino F:\JIVillL\openneuro\ds006072.
4. **Incremental seguro**: al saltar de `t0` a `t1`, el chunk **añade** los runs nuevos sin tocar los ya copiados (y ya denoised por NORDIC). La idempotencia del chunk NORDIC (`DenoisingMethod == "NORDIC"` en el sidecar) salta los runs ya procesados. Ningún ciclo de NORDIC desperdiciado al escalar de tier.

El toggle `REBUILD_FROM_SCRATCH` permite forzar una reconstrucción limpia si la copia del tier quedase en estado inconsistente (p. ej., NORDIC abortado a mitad de run) — opción destructiva que fuerza re-ejecutar NORDIC sobre todo el tier.

El canary ya procesado (sub-P4 ses-6 BOLDREST1 run-1, fila 5 de Tier 0) está en el dataset fuente `F:\` sin marcar como NORDIC (se revirtió tras validación; ver [revert_nordic_canary.py](#)), así que al copiarse al tier se re-procesará por NORDIC con el mismo chunk — buen chequeo de estabilidad del pipeline.

```
# =====
# Construcción del BIDS filtrado en prep/nordic/data/<tier> para el tier activo.
#
# Por cada sesión con runs seleccionados en TIER_RUNS[TIER] copia:
#   - Archivos BIDS raíz (dataset_description.json, participants.tsv, etc.)
#   - sub-PX/ses-Y/anat/*      (requerido por fMRIPrep; no filtrable por run)
#   - sub-PX/ses-Y/fmap/*      (pepolar B0; referenciado por B0FieldSource)
#   - sub-PX/ses-Y/func/*      SÓLO los archivos del run (mag+phase+json,
#                               todos los ecos); y los sidecars task-level.
#
# Política por defecto (REBUILD_FROM_SCRATCH=False):
#   - Copia sólo archivos ausentes en destino. Preserva cualquier run
#     ya denoised por NORDIC (NORDIC sobrescribe mag in-place), que la
#     idempotencia del chunk NORDIC saltará en la siguiente ejecución.
#   - Al escalar de tier: añade los runs nuevos sin re-copiar los viejos.
#
# Política destructiva (REBUILD_FROM_SCRATCH=True):
#   - Borra prep/nordic/data/<tier> entero y recopia desde el fuente
#     pristino F:\. Fuerza re-ejecutar NORDIC sobre todos los runs del
#     tier. Úsese sólo si el estado de la copia del tier quedó inconsistente.
#
# Dependencias: shutil, pathlib (stdlib). No requiere oct2py ni nibabel.
# =====
import shutil
import functools
from pathlib import Path

print = functools.partial(print, flush=True)

SRC_BIDS = Path("F:/JIVillL/openneuro/ds006072")
TIER      = "t2"    # ← cambiar a "t1" para procesar el segundo tier
# La copia filtrada de cada tier va a su subcarpeta bajo prep/nordic/data/,
# donde NORDIC (PsiloPrep S7) la procesa in-place. Así el output denoised
# queda ya organizado por tier para el QC de S7.9 y las etapas siguientes.
```

```

DST_BIDS = Path("D:/ProfessionalProjects/PsyConn/derivatives/ds006072/prep/nordic/data") / T

# Si True: borra DST_BIDS antes de copiar. Pierdes el output NORDIC de
# runs ya procesados (forzaría re-ejecución). Úsese con cuidado.
REBUILD_FROM_SCRATCH = False

# -----
# Definición de los tiers (lista de (subj, ses, task, run))
# -----
TIER_RUNS = {
    "t0": [
        ("sub-P1", "ses-6", "BOLDREST1", "run-1"),
        ("sub-P1", "ses-11", "BOLDREST1", "run-1"),
        ("sub-P3", "ses-8", "BOLDREST1", "run-1"),
        ("sub-P3", "ses-12", "BOLDREST1", "run-1"),
        ("sub-P4", "ses-6", "BOLDREST1", "run-1"),
        ("sub-P4", "ses-10", "BOLDREST1", "run-1"),
        ("sub-P5", "ses-6", "BOLDREST2", "run-1"), # BOLDREST1 run-1 excluido en QC
        ("sub-P5", "ses-10", "BOLDREST2", "run-1"), # BOLDREST1 run-1 excluido en QC
        ("sub-P6", "ses-5", "BOLDREST1", "run-1"),
        ("sub-P6", "ses-9", "BOLDREST3", "run-1"), # BOLDREST1/2 EXCLUDED (pre-flight)
        ("sub-P7", "ses-7", "BOLDREST2", "run-1"),
        ("sub-P7", "ses-11", "BOLDREST2", "run-1"),
    ],
    "t1": [
        ("sub-P1", "ses-6", "BOLDREST1", "run-2"),
        ("sub-P1", "ses-11", "BOLDREST1", "run-2"),
        ("sub-P3", "ses-8", "BOLDREST1", "run-2"),
        ("sub-P3", "ses-12", "BOLDREST1", "run-2"),
        ("sub-P4", "ses-6", "BOLDREST2", "run-1"),
        ("sub-P4", "ses-10", "BOLDREST2", "run-1"),
        ("sub-P5", "ses-6", "BOLDREST1", "run-2"),
        ("sub-P5", "ses-10", "BOLDREST2", "run-2"),
        ("sub-P6", "ses-5", "BOLDREST4", "run-1"),
        ("sub-P6", "ses-9", "BOLDREST4", "run-1"),
        ("sub-P7", "ses-7", "BOLDREST1", "run-1"),
        ("sub-P7", "ses-11", "BOLDREST1", "run-1"),
    ],
}

# Runs procesados por NORDIC pero excluidos manualmente en el control de
# calidad posterior (fMRIPrep / TEDANA): no entran en el set final de análisis.

```

```

#   sub-P5   ses-6   BOLDREST1 run-1
#   sub-P5   ses-10  BOLDREST1 run-1
#   sub-P5   ses-10  BOLDREST1 run-2
#   sub-P7   ses-7   BOLDREST3 run-1
#   sub-P7   ses-7   BOLDREST4 run-1
#   sub-P7   ses-7   BOLDREST6 run-1

selected = TIER_RUNS[TIER]
print(f"Tier activo : {TIER}")
print(f"Runs sel.   : {len(selected)}  (× 5 ecos = {len(selected)*5} run-ecos)")
print(f"Origen      : {SRC_BIDS}")
print(f"Destino       : {DST_BIDS}")
print(f"Rebuild      : {REBUILD_FROM_SCRATCH}")
print("-" * 78)

# -----
# 0. (Opcional) wipe destructivo
# -----
if REBUILD_FROM_SCRATCH and DST_BIDS.exists():
    print(f"[REBUILD] borrando {DST_BIDS} ...")
    shutil.rmtree(DST_BIDS)
DST_BIDS.mkdir(parents=True, exist_ok=True)

# -----
# 1. Helper: copia idempotente (salta si destino ya existe con mismo tamaño)
# -----
def copy_if_missing(src: Path, dst: Path) -> int:
    """Copia src->dst si dst no existe o tiene tamaño distinto.
    Devuelve bytes copiados (0 si saltado)."""
    if dst.exists() and dst.stat().st_size == src.stat().st_size:
        return 0
    dst.parent.mkdir(parents=True, exist_ok=True)
    shutil.copy2(src, dst)
    return dst.stat().st_size

# -----
# 2. Ficheros BIDS raíz (dataset_description.json, participants.tsv, etc.)
# -----
bytes_root = 0
for p in SRC_BIDS.iterdir():
    if p.is_file():
        bytes_root += copy_if_missing(p, DST_BIDS / p.name)

```

```

print(f"root-level : {bytes_root/1e6:7.1f} MB nuevos")

# -----
# 3. Agrupar selección: sujetos únicos (para anat) y sesiones BOLD
#     (para fmap + func filtrado por run).
# -----
subjects_sel = sorted({subj for subj, _, _ in selected})
sessions = {}
for subj, ses, task, run in selected:
    sessions.setdefault((subj, ses), []).append((task, run))

bytes_total = bytes_root

# -----
# 4. Anat - copiar la(s) sesión(es) anatómica(s) completas por sujeto.
#     En ds006072 la anat vive en una sesión dedicada, distinta de las BOLD
#     (ses-1 para sub-P1..P6, ses-0 para sub-P7). Por eso NO se encuentra
#     iterando sessions[] (que sólo contiene sesiones BOLD del tier):
#     hay que descubrirla directamente bajo sub-PX/ses-*/anat.
# -----
print("[anat]")
for subj in subjects_sel:
    bytes_subj = 0
    anat_dirs = sorted((SRC_BIDS / subj).glob("ses-*/anat"))
    if not anat_dirs:
        print(f"    {subj}: no se encuentra carpeta anat en ninguna ses-")
        continue
    for anat_src in anat_dirs:
        anat_ses = anat_src.parent.name # p. ej. 'ses-1' (o 'ses-0' para P7)
        anat_dst = DST_BIDS / subj / anat_ses / "anat"
        for f in anat_src.rglob("*"):
            if f.is_file():
                bytes_subj += copy_if_missing(
                    f, anat_dst / f.relative_to(anat_src))
    bytes_total += bytes_subj
    sess_tag = ",".join(d.parent.name for d in anat_dirs)
    print(f"    {subj} anat ({sess_tag}): nuevo={bytes_subj/1e9:6.2f} GB")

# -----
# 5. Por sesión BOLD: fmap completo + func filtrado por run.
# -----
print("[fmap + func]")

```

```

for (subj, ses), runs_in_ses in sorted(sessions.items()):
    src_ses = SRC_BIDS / subj / ses
    dst_ses = DST_BIDS / subj / ses
    bytes_ses = 0

    # -- fmap completo (pepolar SpinEchoFieldMap AP+PA, ref. por B0FieldSource) --
    fmap_src = src_ses / "fmap"
    if fmap_src.exists():
        for f in fmap_src.rglob("*"):
            if f.is_file():
                bytes_ses += copy_if_missing(
                    f, dst_ses / "fmap" / f.relative_to(fmap_src))

    # -- func: sidecars task-level + ficheros de los runs seleccionados --
    func_src = src_ses / "func"
    func_dst = dst_ses / "func"
    if func_src.exists():
        for p in func_src.glob("task-*_bold.json"):
            bytes_ses += copy_if_missing(p, func_dst / p.name)
        for task, run in runs_in_ses:
            # Patrón: 5 ecos × (mag + phase en complex / sólo mag en
            # mag-only) + sus JSONs = 10 (mag-only) o 20 (complex) ficheros.
            # Excluimos los backups .pre-nordic de los canarys, que no
            # son parte del dataset fuente oficial.
            pattern = f"{subj}_{ses}_task-{task}_dir-*_{run}_"
            hits = [h for h in func_src.glob(pattern)
                    if not h.name.endswith(".pre-nordic")]
            if not hits:
                print(f"    {subj} {ses} {task} {run}: sin matches en origen")
                continue
            for p in hits:
                bytes_ses += copy_if_missing(p, func_dst / p.name)
            n_mag = sum(1 for h in hits
                        if "_part-mag_" in h.name and h.suffixes[-2:] == [".nii", ".gz"])
            n_phase = sum(1 for h in hits
                          if "_part-phase_" in h.name and h.suffixes[-2:] == [".nii", ".gz"])
            print(f"    {subj} {ses} {task} {run}: "
                  f"{n_mag} mag + {n_phase} phase niftis (+ jsons)")

    bytes_total += bytes_ses
    print(f"    {subj} {ses}: {len(runs_in_ses)} run(s)  nuevo={bytes_ses/1e9:6.2f} GB")

```

```
print("-" * 78)
print(f"Total copiado (nuevo) : {bytes_total/1e9:.2f} GB en {DST_BIDS}")
print(f"Ficheros ya presentes fueron saltados (idempotente).")
```